

**Faculty of Engineering Department of
Informatics Engineering-Software and
Information Systems Engineering**



Project-Based Programming Education System

A senior 2 project report

Prepared by:

Malakeh Harah

Raghad Laila

Supervised by:

Eng. Anas Abdulaziz

2025_2026

نظام تعليم البرمجة القائم على المشاريع

Project-Based Programming Education System

تقرير مشروع تخرج 1 - مقدم لاستكمال متطلبات الحصول على درجة البكالوريوس في هندسة المعلوماتية

إعداد الطلاب:

رغد ليلي

ملكة حرح

إشراف

م.أنس عبد العزيز

2026-2025

الإهداء

نتقدم بخالص الشكر و التقدير و الامتنان إلى الجامعة السورية الخاصة و إلى إدارة كلية هندسة الذكاء الاصطناعي على توفير بيئة أكاديمية متميزة .
كما يطيب لنا أن نتقدم بالكثير من الشكر و الامتنان لمشرفنا الدكتور أنس عبد عزيز على توجيهاته السديدة و دعمه المستمر طوال فترة إنجاز هذا المشروع .
ولا ننسى أن نتوجه بالشكر لكل من ساهم في إنجاح هذا العمل من قريب أو بعيد .

Contents

ملخص.....	11
Abstract.....	12
Section One: Project Introduction.....	13
Chapter 1: General Introduction.....	14
1.1 Introduction :	15
1.2 Project Overview and Background:.....	15
1.3 Problem Statement :	15
1.4 Project Objectives :	16
1.5 Contribution to Knowledge :	17
1.6 Research Methodology :	17
1.7 Document Structure :	18
1.8 Summary :	19
Chapter 2: Project Development and Management Methodology	20
2.1 Introduction :	21
2.2 Selected Development Methodology and Rationale	21
2.3 Project Timeline :	21
2.4 Risk Management:	22
2.5 The Team Is Diverse And The Roles Are Varied :	23
2.6 Quality Management :	23
2.7 Summary :	23
Section Two: Future Study and Analysis.....	25
Chapter 3: Review of previous literature and studies	26
3.1 The reference study:	27
3.2 Modern techniques and frameworks used :	28
3.3 Research Gap and Proposed Innovation :	28
3.4 Proposed Innovation :	28
3.5 Summary :	29
Chapter 4 : Requirements Modelling And Analysis	30
4.1 Feasibility Study :	31

4.2 Gather requirements :.....	31
4.2.1 System Data Collection and Gap Analysis :	31
4.3 Functional Requirements :	31
4.3.1 Account Management :	31
4.3.2 Learning Path Management :	32
4.3.3 Project Management :	32
4.3.4 User experience :	34
4.3.5 Alerts system :	35
4.3.6 Store management :	35
4.3.7 Artificial intelligence features :	35
4.3.8 Dashboard :	35
4.3 Non Functional Requirements	35
4.4 UML Schemes for Requirements :	37
4.4.1 Use Case Diagrams:	37
4.4.2 System features – use case specifications (Activity diagrams - Sequence diagrams) :	38
Test Case Description :	38
Introduction:	38
Use Case Specification:	38
Password Reset (UC-1) :	38
Password Reset_ Activity diagram :	39
Password Reset_ Sequence diagram :	40
Adaptive Level Assessment (UC-02) :	40
Adaptive Level Assessment _ Activity diagram :	41
Adaptive Level Assessment _ Sequence diagram :	43
AI Guidance System (UC-03):	43
AI Guidance System _ Activity diagram :	45
AI Guidance System _ Sequence diagram :	46
AI Code Improvement & Quality Review (UC-04) :	46
AI Code Improvement & Quality Review_ Activity diagram :	48

AI Code Improvement & Quality Review_ Sequence Diagram:	49
Virtual Branches Management (UC-05):.....	49
Virtual Branches Management_ Activity diagram :.....	50
4.5 Modeling According to the Selected Methodology :	51
Increment 1: Core System Features :	51
Increment 2: Advanced Learning and Project Features:.....	51
Increment 3: Intelligent Feedback and Analytics Features :	52
4.6 Complete Requirements Traceability Matrix (RTM) :	53
4.7 Initial test cases :	57
Sce-01: Login functionalities :	57
Sce-02 : Create account functionalities :	57
Sce-03: Profile Management functionalities.....	58
Sce-04: Add Learning Path functionalities :	59
Sce-05: Edit Learning Path functionalities :	60
Sce-06: Delete Learning Path functionalities :	60
Sce-07: View Learning Path functionalities :	60
Sce-08: Join Learning Path functionalities :	61
Sce-09: Learner Level Assessment functionalities :	61
Sce-10 : Add Project functionalities :	62
Sce-11: Edit Project functionalities :	62
Sce-12: Delete Project functionalities :	63
Sce-13: View Projects functionalities :	63
Sce-14: Start Project functionalities :	64
Sce-15: Code Editor and Execution functionalities :	64
Sce-16: Hints functionalities :	65
Sce-17: Project Submission and Evaluation :	65
Sce-18: Dashboard functionalities :	66
Sce-19: Automated Code Testing functionalities :	66
Sce-20: Supervisor Evaluation functionalities :	67

Sce-21: Notifications functionalities :	67
Sce-22: Isolated Code Sandbox functionalities	67
4.8 Summary:	67
Section three: Design and architecture	69
Chapter 5 : System architectural design	70
5.1 High-Level Design :	71
5.1.1 System Block Diagram :	71
5.1.2 Architectural design style used :	72
5.1.3 Architectural Pattern & System Architecture Design:	73
5.2 DataBase Design :	74
5.2.1 ER Diagram :	74
5.2.2 Relational Schema :	75
5.2.3 Constraints and Keys:	76
5.3 External APIs Design :	76
5.4 Summary :	77
Section four: Implementation and testing.....	78
Chapter 6 : Execution environment and tools	79
6.1 Tools Used :	80
6.2 Development Environment and Project Structure :	80
6.3 Programming Languages and Frameworks :	80
6.4 Database Server and Operating System :	80
6.5 Dependencies :	80
6.6 Version Control and Source Code Management :	81
6.7 Continuous Integration and Deployment (CI/CD):	81
6.8 Testing Environment :	81
6.9 Summary :	81
Chapter 7 : Programmatic implementation	82
7.1 Project Structure :	83
7.2 Modules Implementation	84
7.3 API Implementation :	85

7.4 Data Access Layer Implementation :	86
7.5 Business Logic Layer :	88
7.6 User Interface Implementation :	88
Login Page :	89
Register Page:	89
Learner Dashboard :	90
Learning Paths Page :	91
Project Details Page :	91
Instructor Dashboard / Project Management Page :	92
Placement test page:	92
Code Editor :	93
AI-powered code :	96
7.7 Intelligent Algorithms Implementation :	97
Chapter 8 : Testing and evaluation	99
8.1 Test Plan:	100
8.2 Types Of Testing :	100
8.3 Test Case Diagrams :	101
8.3.1 Test Case Diagrams :	101
8.3.2 Test Results :	105
8.3.3 Test Results and Analysis :	105
8.4 Summary:	106
Section five : Results and recommendations	107
Chapter 9 : Results and analysis	108
9.1 System implementation results :	109
9.2 Comparative Analysis with Existing Systems:	118
This section presents a comprehensive comparison between the proposed system and existing programming learning platforms, taking into account recent enhancements such as AI-driven code reviews and virtual workspace branching.	118
9.3 Achievement of Objectives:	119
9.4 Strengths and Weaknesses Analysis :	119

9.5 Summary:	120
Chapter 10 : Conclusion and Recommendations	121
10.1 Project Summary and Main Achievements.....	122
10.2 Challenges and Difficulties:	122
10.3 Future Work:	123
10.4 Recommendations:	123
10.5 Summary :	123
Section Six: Appendices and References	125
Section A: Requirements Gathering & Gap Analysis	126
A.1 Appendix Introduction :	127
Section B: Detailed Design	128
B.1 Class & Component Diagrams :	129
B.1.1 Detailed Class Diagram and Relationships :	129
B.1.2 Component Diagram :	129
B.2 Modules Design :	129
B.3 APIs Design :	130
B.4 Smart Algorithms Design :	130
B.4.1 Data Flow Diagrams (DFD) & State Machine Diagrams :	130
B.4.2 Algorithmic Pseudo-code.....	131
B.5 UI/UX Design & System Screenshots :	131
Appendix C: Detailed Test Results	132
C.1 Introduction to System Testing :	133
C.2 Test Logs & Execution Summary :	133
C.3 Test Snapshots & Verification Logs :	134
Appendix D: User Manual & Installation Guide	135
D.1 Introduction :	136
D.2 System Prerequisites & Environment Requirements :	136
D.3 System Installation & Setup :	136
D.4 System Operation & User Guide :	137
References	138

ملخص

في عصر التقدم التقني المتسارع والاعتماد المتزايد على الحلول الرقمية، أصبحت المنصات الإلكترونية أدوات حيوية لإدارة وتعزيز العمليات التجارية والخدمية. يقدم هذا المشروع نظاماً رقمياً شاملاً مصمماً لسد الفجوة بين المعرفة النظرية والتطبيق العملي، حيث يوفر بيئة تطوير متكاملة مدعومة بتقنيات الذكاء الاصطناعي التي تقدم توجيهاً فورياً للمتعلمين، وتقييماً أدائياً ذكياً يواكب مستواهم المهاري. يعتمد النظام على بنية برمجية متطورة تتيح للمتعلم بناء مشاريع حقيقية ضمن مسارات تعليمية مخصصة، مما يضمن اكتساب خبرة عملية تؤهلهم لسوق العمل بفاعلية، يهدف النظام إلى توفير منصة سهلة الاستخدام وأمنة وفعالة، في هذا التقرير، نقدم لمحة عن متطلبات النظام، الوظيفية وغير الوظيفية، وتصميمه المعماري، وتفاصيل التنفيذ، ونتائج الاختبارات. وتتبع أفضل الممارسات في هندسة البرمجيات لضمان قابلية التوسع، وسهولة الصيانة، وتجربة مستخدم سلسة.

Abstract

"Buildly" is an innovative educational platform specifically designed to facilitate programming education through the Project-Based Learning (PBL) methodology. The system aims to bridge the gap between theoretical knowledge and practical application by providing an integrated development environment (IDE) enhanced with AI-driven capabilities. These features offer real-time guidance and intelligent performance assessment tailored to each learner's skill level. By leveraging a robust software architecture, Buildly enables students to build real-world projects within personalized learning paths, ensuring the acquisition of practical experience that effectively prepares them for the software industry.

Section One: Project Introduction

Chapter 1: General Introduction

1.1 Introduction:

The technical education sector is witnessing radical transformations driven by rapid technological advancements, where traditional education alone can no longer meet the growing industry demand for practical programming skills. In this context, Project-Based Learning (PBL) emerges as an effective pedagogical methodology to bridge the gap between academic knowledge and professional practice. This project introduces "**Buildly**", an innovative and intelligent learning environment designed to transform the programming learning experience by integrating artificial intelligence to provide personalized guidance and adaptive learning paths. This chapter outlines the project's foundational framework, defining the strategic objectives, research methodology, and scientific contributions, setting the stage for exploring the development and analysis phases in the following chapters.

1.2 Project Overview and Background:

In light of the rapid advancements in the information technology sector, transitioning from traditional theoretical education to practical Project-Based Learning (PBL) has become an essential necessity to prepare a generation of developers capable of meeting the demands of the labor market. Current programming education often relies on methods that lack direct guidance and intelligent assessment tailored to individual differences among learners, which leads to a significant gap between academic skills and actual programming practices.

1.3 Problem Statement:

The fundamental problem in current programming education lies in relying excessively on theoretical content or isolated programming exercises, which leads to the following challenges:

- **Weak Practical Integration:** Learners struggle to bridge the gap between theoretical knowledge and real-world project development, as traditional environments lack step-by-step guidance.
- **Lack of Intelligent Feedback:** Current learning systems suffer from slow or absent immediate, personalized assessment, forcing learners to wait for human review, which hinders self-learning and reduces motivation.

The importance of addressing this problem lies in its impact on the core 'skill gap' in the software job market. Developing a platform that provides 'AI-based guidance' does not only save time and effort for the learner, but also acts as a 24/7 digital

mentor, accelerating the learning pace and boosting the learner's confidence in solving complex programming problems. Academically, this project contributes to providing a practical model for integrating artificial intelligence as an augmenting tool in digital learning environments, thereby enhancing the overall quality and efficiency of the educational process.

1.4 Project Objectives:

The project aims to revolutionize programming education. Our objectives are defined according to the SMART methodology (Specific, Measurable, Achievable, Relevant, and Time-bound) as follows:

➤ **Main Objective:**

- Develop an interactive, Project-Based Learning (PBL) platform that integrates artificial intelligence to provide personalized guidance and adaptive learning paths for software engineering students.

➤ **Sub-Objectives:**

- **Intelligent Assessment Engine:** Achieve an accuracy rate of no less than 85% in providing automated feedback and debugging assistance to learners.
- **User Experience (UX) Optimization:** Design an interactive user interface that ensures seamless navigation and access to all project tools in fewer than 3 clicks.
- **Performance Efficiency:** Ensure a system response time (latency) not exceeding 3 Minutes when analyzing the user-submitted programming code.
- **Facilitating Self-Learning:** Provide comprehensive learning paths for three fundamental programming languages (or technologies) by the end of the project development period, ensuring the acquisition of applicable practical skills.
- **Project Management:** Develop a dashboard that enables students to track their progress in project construction and directly manage code repositories (GitHub Integration).

1.5 Contribution to Knowledge:

The project goes beyond being a traditional educational platform; it offers a significant contribution to the field of software engineering education through the following aspects:

- **Intelligent AI Integration in PBL (AI-Integrated PBL):** The project presents a unique model that integrates artificial intelligence techniques, not merely to provide solutions, but to function as a "digital tutor" that analyzes the student's problem-solving approach and provides real-time, personalized guidance tailored to their skill level—a synergy rarely seen in existing Project-Based Learning (PBL) platforms.
- **Adaptive Assessment Framework:** The project contributes an innovative adaptive assessment algorithm. The difficulty level of programming challenges and evaluations is dynamically adjusted based on the student's actual performance (e.g., response time and error rate), ensuring that the learner remains in the "Optimal Challenge Zone" throughout their educational journey.
- **Real-World Integrated Development Environment:** The project bridges the gap between academia and industry by connecting learning paths directly to real-world project management tools (such as direct GitHub integration). This establishes the platform as a hybrid "development and training environment," representing an advanced practical case study on integrating professional tools within the educational process.

1.6 Research Methodology:

This thesis adopts a methodical approach that combines engineering research with experimental evaluation to ensure the delivery of a comprehensive and reliable technical solution. The methodology is structured as follows:

- **Engineering Method:** This is the core methodology employed in this project. The platform development process follows a structured engineering lifecycle, including problem analysis, requirements engineering, architectural design, and system implementation and testing. This method aims to construct "Buildly" as a practical technical solution that effectively addresses the programming skill gap.

- **Experimental Method:** This approach is utilized during the system evaluation phase. The performance of the AI algorithms and the intelligent assessment system is tested under various conditions. This involves measuring specific metrics (such as assessment accuracy, response time, and user experience) and benchmarking the results against predefined targets (SMART Goals).
- **Descriptive Analytical Method:** This method is used to document and analyze existing educational theories and frameworks (such as the PBL methodology) and compare them with the proposed model. This allows us to highlight the qualitative contributions and innovative value that our project introduces in the context of intelligent e-learning.

1.7 Document Structure:

This thesis is structured into ten sequential chapters to provide a comprehensive view of the project, ranging from theoretical foundations to practical application, as follows:

1. Chapter 1 (General Introduction): Establishes the framework of the project, outlining the problem, objectives, and research methodology.
2. Chapter 2 (Project Development and Management Methodology): Reviews the management and software development methodologies adopted to ensure efficient workflow.
3. Chapter 3 (Literature Review): Discusses relevant research and existing systems that have paved the way for this work.
4. Chapter 4 (Requirements Modelling and Analysis): Details the functional and non-functional requirements of the system and their analysis.
5. Chapter 5 (System Architectural Design): Covers the architectural diagrams and the structural design of the system.
6. Chapter 6 (Implementation Environment and Tools): Reviews the software tools and technologies used in building the platform.
7. Chapter 7 (Software Implementation): Highlights the process of transforming designs into effective code.
8. Chapter 8 (Testing and Evaluation): Presents the testing plans to ensure system quality and performance.
9. Chapter 9 (Results and Analysis): Analyzes the outputs and findings reached based on the conducted evaluations.

10. Chapter 10 (Conclusion and Recommendations): Provides a summary of the final findings, future recommendations, and potential project enhancements.

1.8 Summary:

This chapter establishes the foundational framework for the project, identifying the core challenge in programming education: the gap between theoretical knowledge and practical application. Consequently, project objectives were formulated using the **SMART** methodology to ensure the development of an intelligent, Project-Based Learning (PBL) platform that provides real-time guidance and adaptive learning paths. The chapter also highlighted the project's qualitative contribution in presenting a hybrid model that integrates artificial intelligence with professional development tools, grounded in a research methodology combining engineering, experimental, and descriptive approaches. Finally, this chapter provided a structured roadmap for the thesis, ensuring a comprehensive focus on all project stages, from initial analysis to final evaluation.

Chapter 2: Project Development and Management Methodology

2.1 Introduction:

Systematic planning and rigorous resource management are the cornerstones of any successful software project. In this chapter, we present the administrative and methodological framework adopted for the implementation of the Buildly platform. We transition here from the theoretical vision and objectives established in the first chapter to practical execution. This chapter aims to clarify the structured workflow that ensures the quality of software deliverables and the efficiency of project progress within the defined timeline. It details the selected development methodology and the technical rationale behind it, alongside the project timeline, risk management strategies, the distribution of roles and tasks, and the standards adopted to ensure system quality. This provides a clear roadmap that ensures the transformation of project requirements into a tangible reality.

2.2 Selected Development Methodology and Rationale:

The **Incremental Model** was selected as the core methodology for developing the "Buildly" platform. This model is based on dividing the system into executable parts or "increments" that can be delivered independently. Each increment is developed and tested before proceeding to the next, ensuring the efficiency of the cumulative system construction.

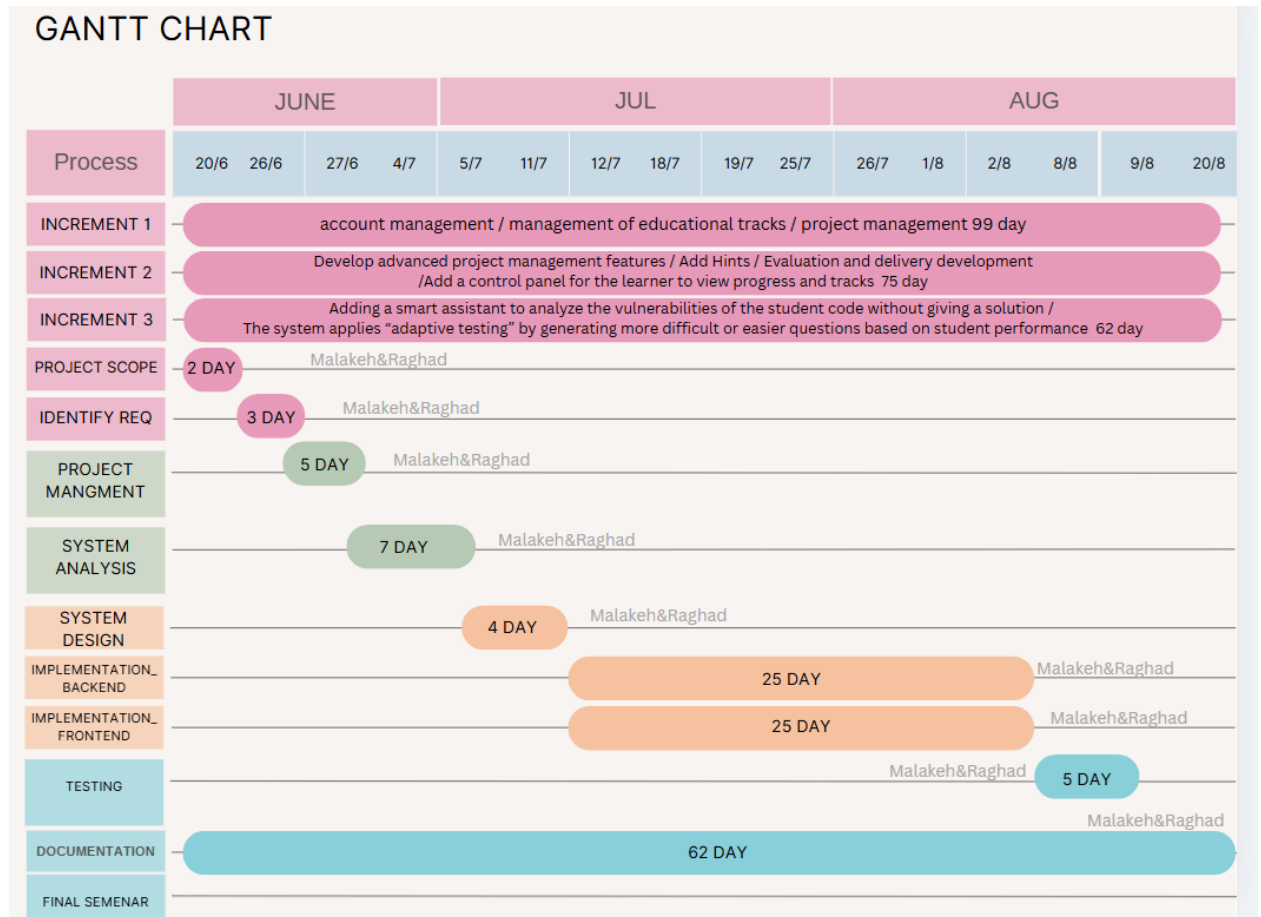
Rationale for Selecting the Incremental Model:

1. **Development Flexibility:** This methodology allows for the integration of complex features (such as the AI engine or adaptive assessments) in phases, without compromising the stability of the core components already built.
2. **Risk Management:** By delivering the system in increments, it becomes easier to detect and resolve programming errors early in each phase, reducing the likelihood of critical failures during final integration.
3. **Continuous Progressive Development:** Since Buildly focuses on diverse learning paths, this methodology enables us to develop each "learning path" as a separate increment, facilitating a process of continuous improvement based on ongoing feedback.

2.3 Project Timeline:

To ensure the execution of the Buildly project within the planned schedule, a Gantt Chart was utilized to illustrate the sequence of tasks and the duration of each incremental development phase. The timeline is designed to allow for task overlap where feasible, enhancing efficiency while allocating sufficient time for testing and documentation. The table above displays the project's temporal

distribution, spanning over [9 weeks], balancing the development of technical features with adherence to academic standards.



2.4 Risk Management:

To ensure project continuity, potential risks were analyzed, and appropriate mitigation strategies were identified:

Risk	Probability	Impact	Mitigation Strategy
AI Model Inaccuracy	Medium	High	Utilize a fallback model and regularly update the training dataset.
GitHub API Integration Issues	High	Medium	Implement robust exception handling and provide manual upload options.

Schedule Overrun	Medium	High	Adhere to the incremental methodology to ensure delivery of core features.
Data Loss	Low	Critical	Execute automated daily database backups.
System Latency	Medium	Medium	Perform code optimization and utilize caching techniques.

2.5 The Team Is Diverse and The Roles Are Varied:

Stakeholder	Role	Responsibility
Eng. Anas Abdulaziz	Project Manager & Supervisor	Guides the project and provides feedback.
Raghad Laila	Software Engineer	Develop backend functionalities, implement authentication, handle database integrations and documentation.
Malakeh Harah	Software Engineer	Frontend development, ensure a seamless user experience and documentation.

2.6 Quality Management:

Quality management is an essential aspect of the software development process, as it ensures that the final system meets the required standards and satisfies user expectations. In this project, several practices and strategies are adopted to maintain and improve the quality of the Project Based Programming Education System throughout its development lifecycle.

2.7 Summary:

This chapter presented the selected development methodology, project planning, risk management, team roles, and quality

management practices. The Incremental Development Methodology was adopted to support gradual system development and testing. These management and planning activities helped organize the development process and ensure project quality.

Section Two: Future Study and Analysis

Chapter 3: Review of previous literature and studies

3.1 The reference study:

feature	Code cade my	Free Code Camp	Solo Learn	The Odin Project	Our system
Code Editor	✓	✓	✓	✗	✓
Automated project evaluation	✓	✓	✓	✗	✓
Using artificial intelligence for guidance	✗	✓	✗	✗	✓
Supporting the experimental environment (Virtual) (Branches	✗	✗	✗	✓	✓
Competitive gaming system	✗	✗	✓	✗	✗
Forcing the learner to build the local environment	✗	✗	✗	✓	✗
Microlearning via a mobile app	✓	✗	✓	✗	✗
Attach starter files containing the code structure.	✓	✓	✗	✓	✓
Projects of increasing difficulty within the educational track	✗	✓	✗	✓	✓
Automatically save project status	✗	✗	✗	✗	✓

3.2 Modern techniques and frameworks used:

Programming languages: CSS, Html, JavaScript.

Framework: React(frontend), Django(backend).

Database: PostgreSQL.

3.3 Research Gap and Proposed Innovation:

Based on the analysis of existing platforms such as "Code cade my", "Free Code camp", Solo Learn, and The Odin Project, several limitations are identified. Most platforms do not provide a fully integrated project-based learning experience, and they lack adaptive learning that adjusts content according to the learner's level. Additionally, features such as project dependencies, detailed progress tracking, and manual evaluation with personalized feedback are either missing or not well implemented.

3.4 Proposed Innovation:

The proposed system addresses these gaps by introducing an integrated platform that combines project-based learning, initial level assessment, structured learning paths with dependencies, and a built-in code editor. It also provides automatic and manual evaluation, progress tracking, and incremental hints, offering a more effective and practical learning experience.

3.5 Summary:

This chapter reviewed previous studies and existing programming learning platforms. It also discussed the technologies and frameworks used in the project and highlighted the research gap addressed by the proposed system. The chapter demonstrated how the project provides a more practical and integrated project-based learning experience.

Chapter 4: Requirements Modelling and Analysis

4.1 Feasibility Study :

The feasibility study evaluates the possibility of successfully developing the proposed system.

From a technical perspective, the system can be implemented using technologies such as Django, React, which are suitable and available.

From an operational perspective, the system is designed to be user-friendly and easy to use for both learners and instructors.

From a time perspective, the project can be completed within the academic semester by following the incremental development approach.

Overall, the project is feasible and can be successfully implemented within the available resources and constraints.

4.2 Gather requirements :

4.2.1 System Data Collection and Gap Analysis:

Instead of traditional questionnaires, our methodology for requirements gathering focused on 'Observational Gap Analysis'. We observed the workflow of students during their project-based learning and conducted a comparative analysis of existing educational platforms to identify unmet needs.

- **Observation of Student Workflow:**

We observed that students often struggle to manage project tasks and integrate them into the learning process. Furthermore, there is a clear deficiency in tracking progress in project-based learning, leading to inconsistent outcomes.

- **Comparative Analysis:**

We analyzed several prominent educational platforms to assess their functionality. Our comparison revealed a significant gap: most platforms offer content delivery (lectures/videos) but fail to provide a code editor for students to practice.

4.3 Functional Requirements:

4.3.1 Account Management:

FR-1.1: The system must allow the user (learner, trainer) to create a new user account in the system.

FR-1.2: The system must allow the user (learner, trainer) to log in using a username and password.

FR-1.3: The system must allow the user to view and manage profile information.

FR-1.4: The system should allow the user to edit profile information.

FR-1.5: The system must allow the user (learner, trainer) to log out of their account.

FR-1.6: The system allows the user to change the account's profile picture.

FR-1.7: The system allows the user to edit basic information.

FR-1.8: The system offers password recovery via email.

4.3.2 Learning Path Management:

FR-2.1: The system must allow the instructor to create a new learning path, specifying its title, description, and content.

FR-2.2: The system should allow the instructor to modify an existing learning path.

FR-2.3: The system should allow the instructor to delete a learning path.

FR-2.4: The system should display a list of available learning paths to the user.

FR-2.5: The system must allow the learner to view information about a specific path.

FR-2.6: The system must allow the learner to join an educational path.

FR-2.7: The system should display the projects associated with each learning path to the learner when entering a path.

FR-2.8: The system must assess the learner upon joining the course to determine their level (beginner, intermediate, advanced) and present them with suitable projects according to this level before starting any project.

4.3.3 Project Management:

FR-3.1: The system should allow the trainer to add a new project with its details specified (title, objectives, tasks, resources, requirements).

FR -3.2: The system should allow the coach to modify an existing project.

FR-3.3: The system should allow the coach to delete the project.

FR-3.4: The system must allow the user (administrator, learner) to view the list of projects in the system.

- FR-3.5:** The system must allow the user (administrator, learner) to view the list of projects in a specific location.
- FR-3.6:** The system must allow the administrator to view specific project information.
- FR-3.7:** The system should allow the user to select a project and view educational project information.
- FR-3.8:** The system should allow the learner to start a specific project.
- FR-3.9:** The system should allow projects to be displayed according to their status (new, in progress, completed).
- FR-3.10:** The system must allow the addition of a multi-stage project (sequential stages).
- FR-3.11:** The system should allow for the definition of project dependencies so that the learner does not start a project until a previous project has been completed.
- FR-3.12:** The system should allow projects to be filtered by level.
- FR-3.13:** The system should allow the attachment of starter files to each project containing a preliminary code structure.
- FR-3.14:** The system should allow the supervisor to "define" the prerequisites for each project when it is created.
- FR-3.15:** The system should provide a built-in code editor with syntax highlighting for projects.
- FR-3.16:** The system must support the execution of the code within the platform and the display of results.
- FR-3.17:** The system should allow the project status to be saved automatically at specified time intervals.
- FR-3.18:** The system should provide a progress bar that shows the percentage of project completion.
- FR-3.19:** The system must support the display of detailed statistics on start time, completion time, and project time rate.
- FR-3.20:** The system must support displaying the administrator's modification history with the ability to revert to a previous version.
- FR-3.21:** The system should allow the learner to submit the project.

FR-3.22: The system should allow deleting tasks in the project.

FR-3.23: The system should allow adding tasks during project modification.

FR-3.24: The system must support providing incremental hints within the project without revealing the complete solution.

FR-3.25: When a project is being executed, the system should display a list of task titles in the sidebar, highlighting the current task.

FR-3.26: The system should allow the supervisor to manually evaluate the project and add text comments or a star rating.

FR-3.27: The system should display a list of delivered projects to the administrator.

FR-3.28: The system should filter submitted projects based on whether they have been evaluated or not.

FR-3.29: The system should allow the learner to download the educational files attached to the project.

FR-3.30: The system should allow the learner to add new tabs to the project.

FR-3.31: The system should allow the learner to delete unwanted tabs from the project.

FR-3.32: The system must support the execution of multi-file projects and the presentation of the final result in an integrated manner.

FR-3.33: Automatically assess the learner's solution and the honey according to the criteria, while displaying the gaps.

4.3.4 User experience:

FR-4.1: The system must support opening files within the editor in tabs.

FR-4.2: The system allows searching for projects and educational pathways using keywords.

FR-4.3: The system allows the learner to review their activity log to track their progress.

FR-4.4: The system allows students to share their achievements on social media platforms.

FR-4.5: The system offers a "Favorites" feature to save paths and projects for later reference.

4.3.5 Alerts system:

FR-5.1: The system provides instant alerts when a project is accepted or a manual evaluation is added.

4.3.6 Store management:

FR-6.1: The system allows the administrator to archive old learning paths instead of deleting them.

FR-6.2: The system allows the supervisor to view archived learning paths in the supervisor's control panel.

FR-6.3: The system provides advanced filtering and searching for supervisors to access pending students or projects.

4.3.7 Artificial intelligence features:

FR-7.1: The system provides a smart assistant to analyze student code flaws without providing a solution.

FR-7.2: The system allows the learner the option to "improve the code" through AI review.

FR-7.3: The system provides virtual branches to test alternative solutions without changing the original code.

FR-7.4: The system applies "adaptive testing" by generating more or less difficult questions based on the student's performance.

4.3.8 Dashboard:

FR-8.1: The system provides the learner with a control panel that displays the learning paths the learner has enrolled in, the number of projects being implemented and completed in the path, and the number of learning hours.

4.3 Non-Functional Requirements:

NFR-4.3.1: The system must encrypt all passwords using secure hashing algorithms and not save them as plain text to protect user data.

NFR-4.3.2: The system should load any web page (except code editor pages) within a maximum of three seconds for 95% of users when using a normal internet connection.

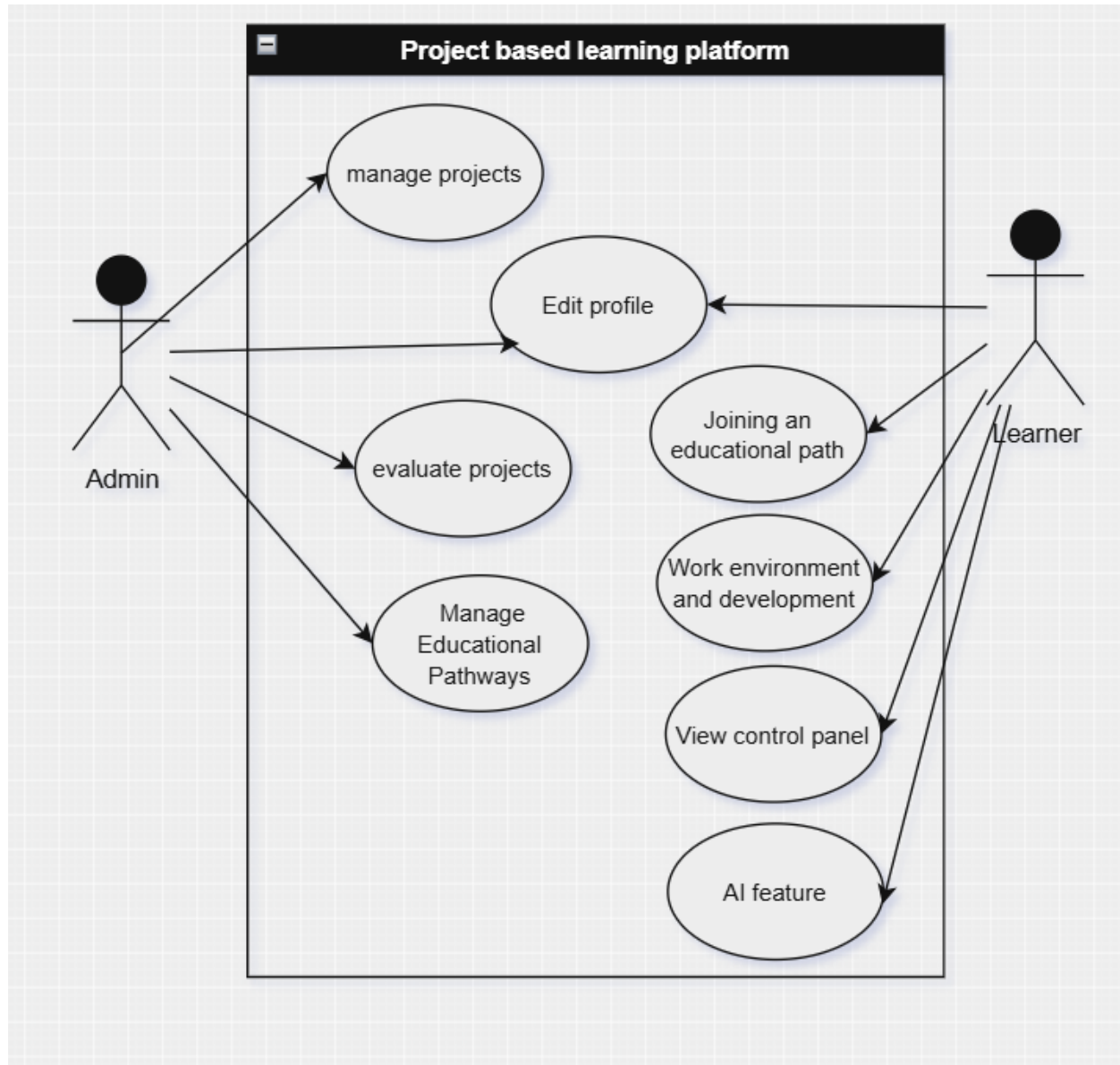
NFR-4.3.3: The system must be scalable to support an increasing number of users without a noticeable drop in performance or the need for major infrastructure changes.

NFR-4.3.4: The system should provide user-friendly interfaces so that a new user needs, on average, "10 minutes" to fully navigate and understand all parts and pages.

NFR-4.3.5: The system should allow the addition of new features or functions with minimal changes to the existing codebase and without affecting the communication of core components with other new services.

4.4 UML Schemes for Requirements:

4.4.1 Use Case Diagrams:



4.4.2 System features – use case specifications (Activity diagrams - Sequence diagrams) :

Test Case Description:

Introduction:

Test cases are used to verify that the system functions correctly according to the defined requirements. They help ensure that each feature operates as expected under different conditions.

In this project, a set of representative test cases is defined to validate the core functionalities of the system, such as account management, learning path operations, and project handling.

These test cases cover the main user interactions and typical scenarios.

Due to the scope of the project, only a subset of test cases is presented in this report, focusing on the most important and frequently used features.

Additional test cases can be defined and executed during further development and testing phases.

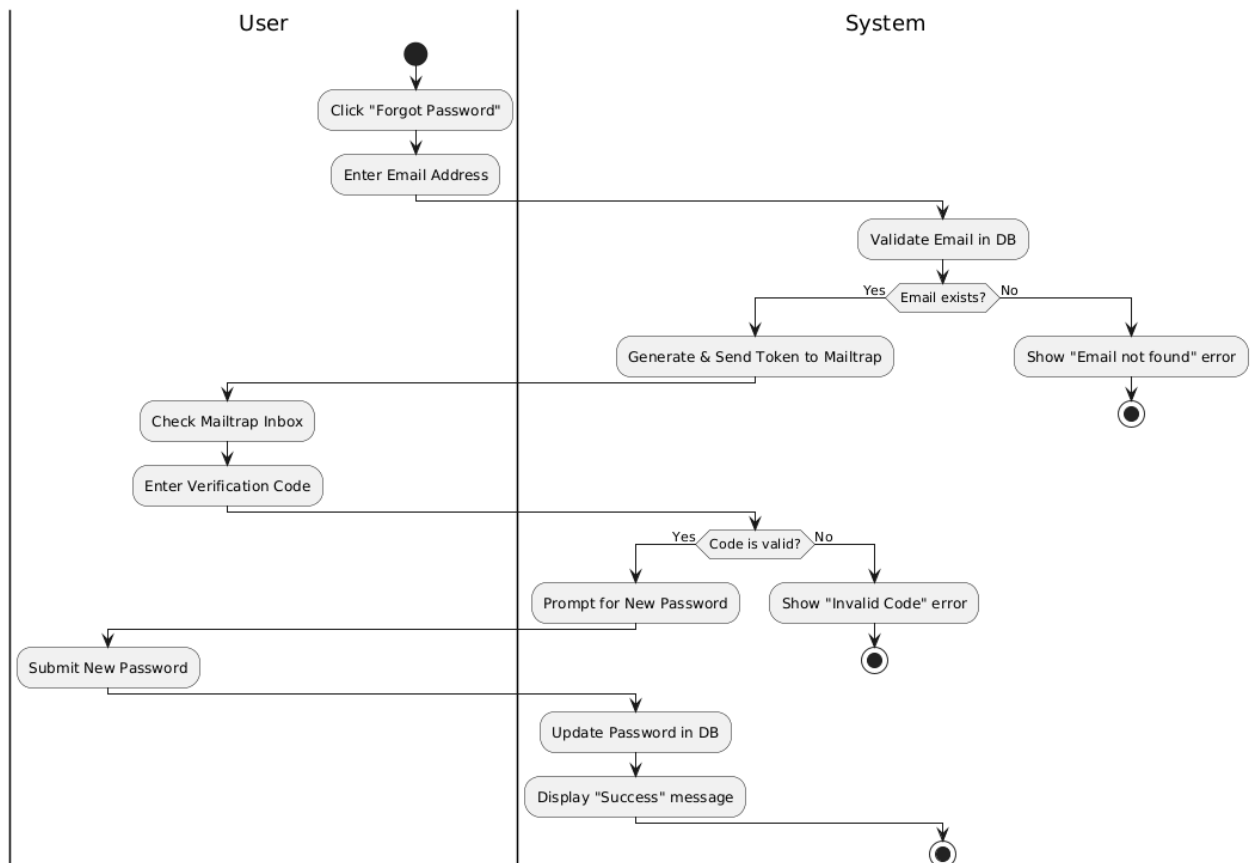
Use Case Specification:

Password Reset (UC-1):

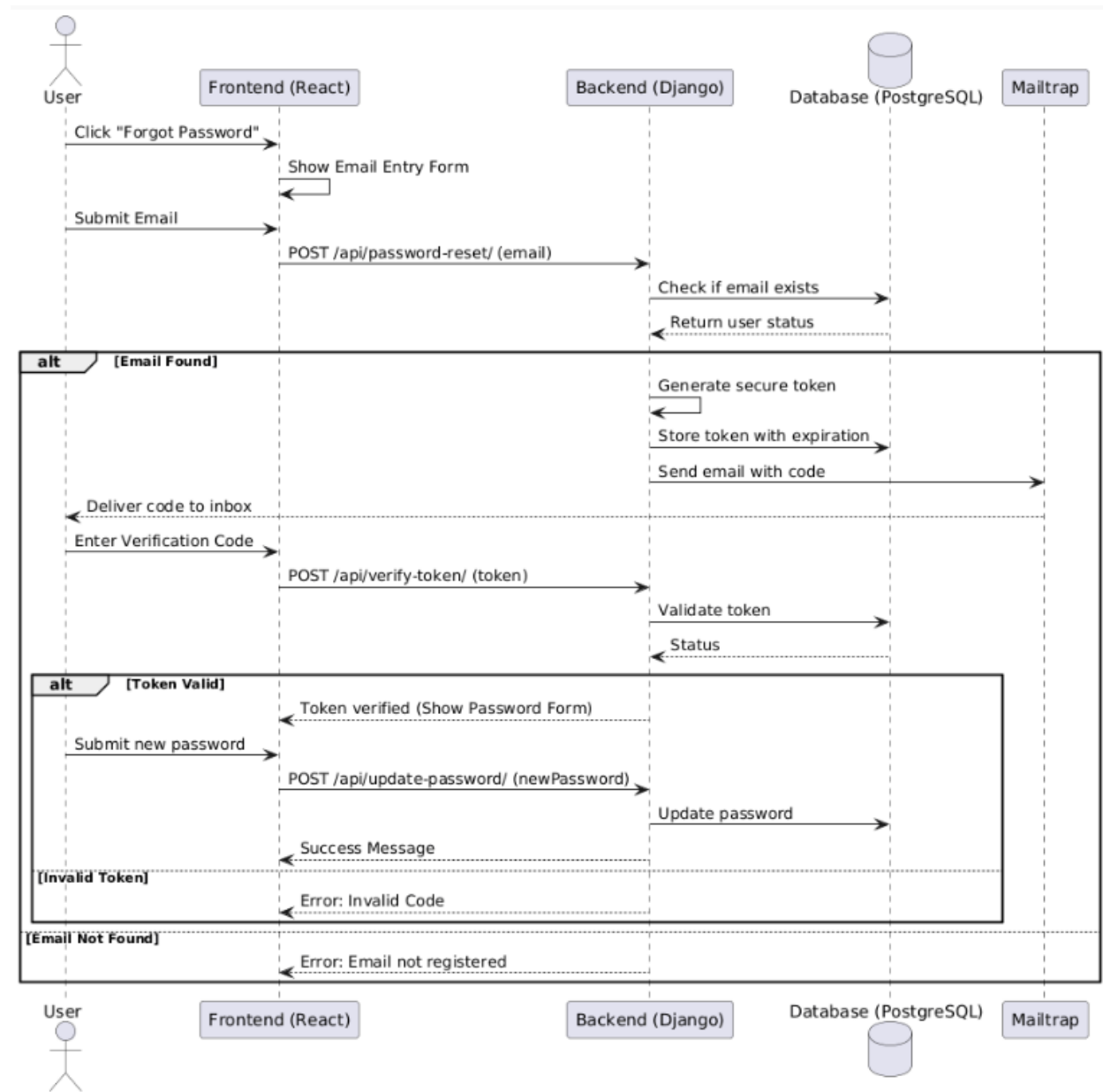
Use Case ID	UC-01
Use Case Name	Password Reset
Actors	User (Student, Admin)
Description	Allows users to regain access to their accounts by requesting a password reset via email verification.
Pre-conditions	1. The user must have a registered account in the system. 2. The system must have a configured SMTP/Mailtrap service for email delivery.
Main Flow	1. The user clicks the "Forgot Password" link on the login screen. 2. The system displays the email entry page. 3. The user enters their registered email address. 4. The system verifies if the email exists in the database.

	5. The system generates and sends a verification token (via Mailtrap). 6. The user enters the verification code on the validation page. 7. Upon successful validation, the system prompts for a new password. 8. The user submits the new password, and the system updates the database.
Alternative Flow	[A1]: Email Not Found: If the email does not exist, the system displays an error: "Email not registered." [A2]: Invalid Token: If the user enters an incorrect code, the system displays an error: "Invalid verification code, please try again."
Post-conditions	The user's password is updated in the system, and they are granted access to their account with the new credentials.

Password Reset_ Activity diagram:



Password Reset_ Sequence diagram:

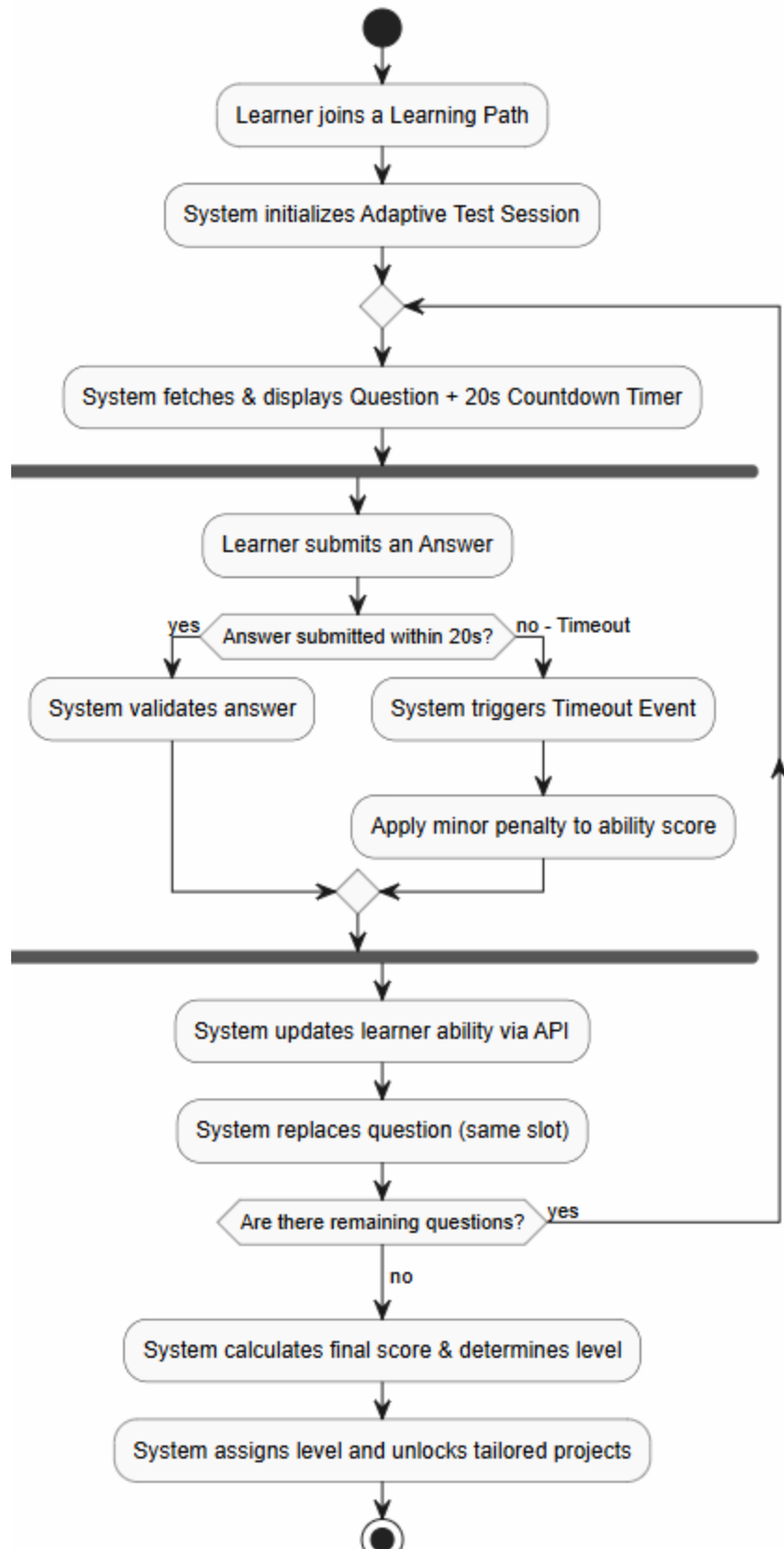


Adaptive Level Assessment (UC-02):

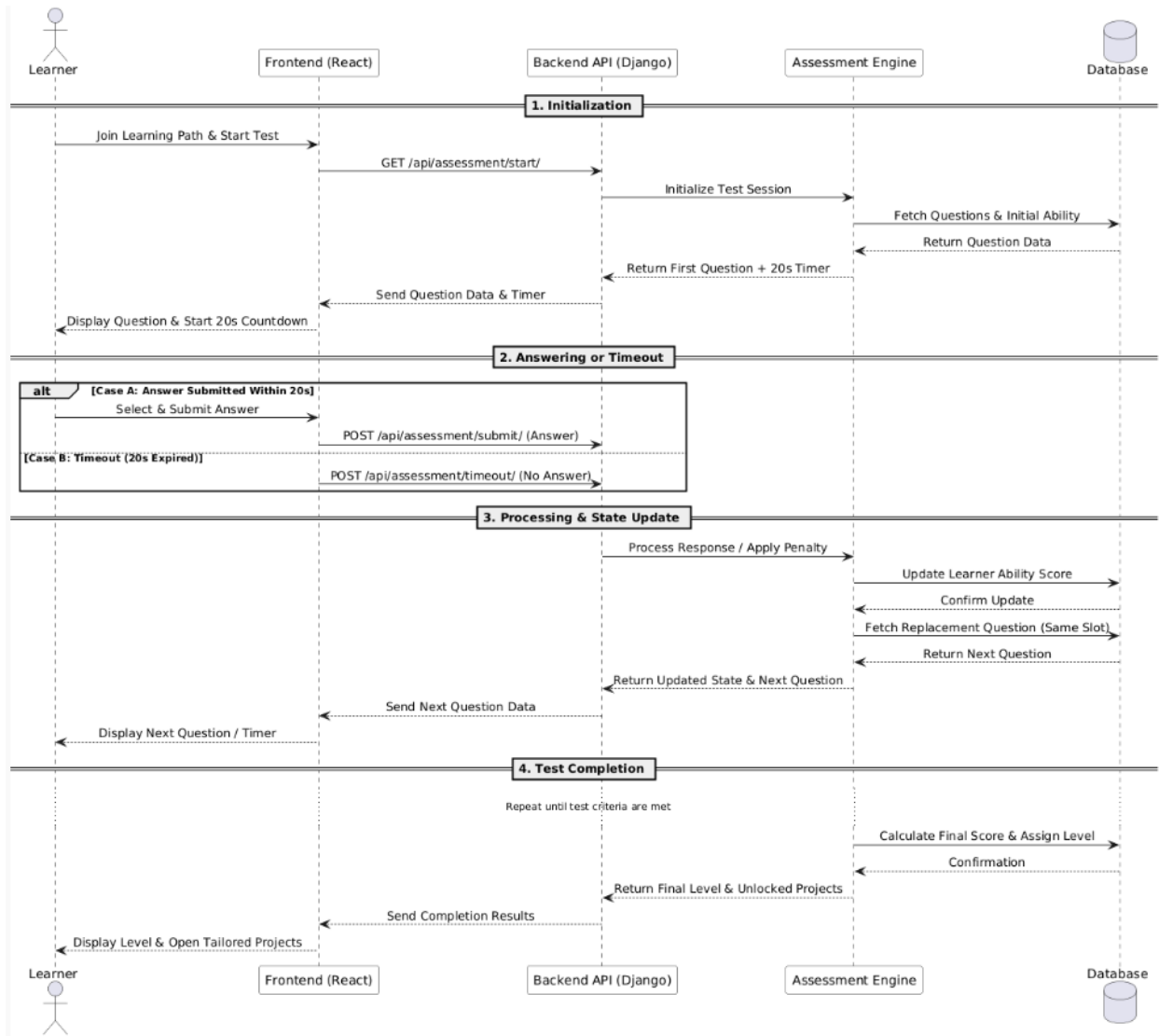
Use Case ID	UC-02
Use Case Name	Adaptive Level Assessment
Actors	New Learner (Student)
Description	Allows a newly registered learner to take an adaptive placement test (for Frontend or Python paths) with dynamic questions, a real-time countdown timer, and automatic skill/ability updates.

Pre-conditions	1. The learner must have an active account and have joined a specific learning path. 2. The assessment question bank for the path must be configured in the system.
Main Flow	1. The learner joins a learning path and triggers the adaptive placement test. 2. The system initializes the test session and displays the first dynamic question along with a 20-second countdown timer. 3. The learner selects an answer and submits it before time runs out. 4. The system validates the answer, updates the learner's ability score via backend APIs, and fetches the next appropriate question. 5. Steps 2-4 repeat until the termination criteria of the test are met. 6. Upon test completion, the system calculates the final score, assigns an appropriate skill level to the learner, and unlocks suitable projects.
Alternative Flow	[A1]: Timeout Exceeded: If the 20-second countdown expires without a user response, the system automatically treats the question as incorrect, applies a minor penalty to the ability score, and replaces the question (maintaining the same question slot/number) via API before moving forward.
Post-conditions	The learner's proficiency level is successfully evaluated and recorded in the database, tailoring their future project roadmap accordingly.

Adaptive Level Assessment _ Activity diagram:



Adaptive Level Assessment _ Sequence diagram:

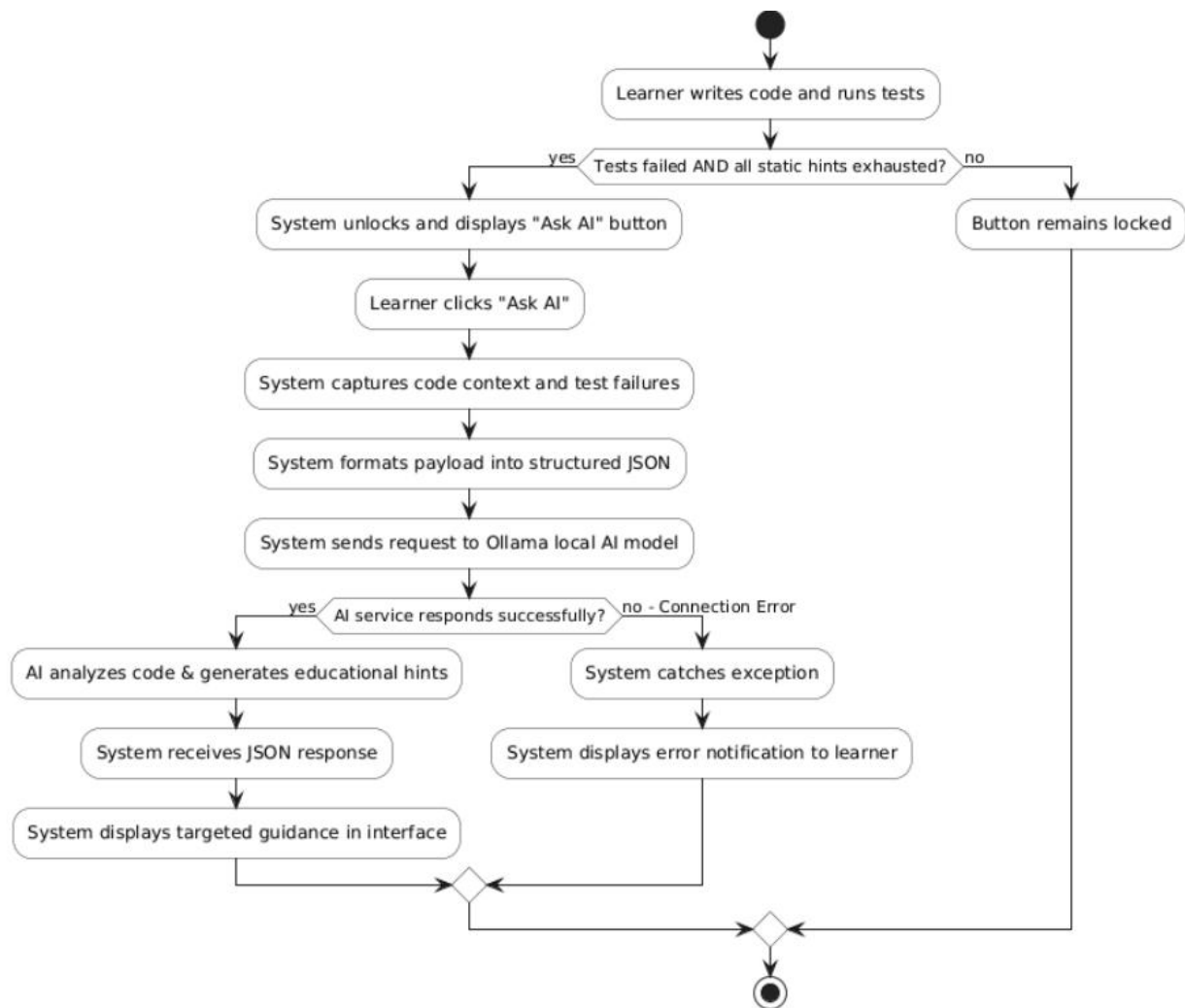


AI Guidance System (UC-03):

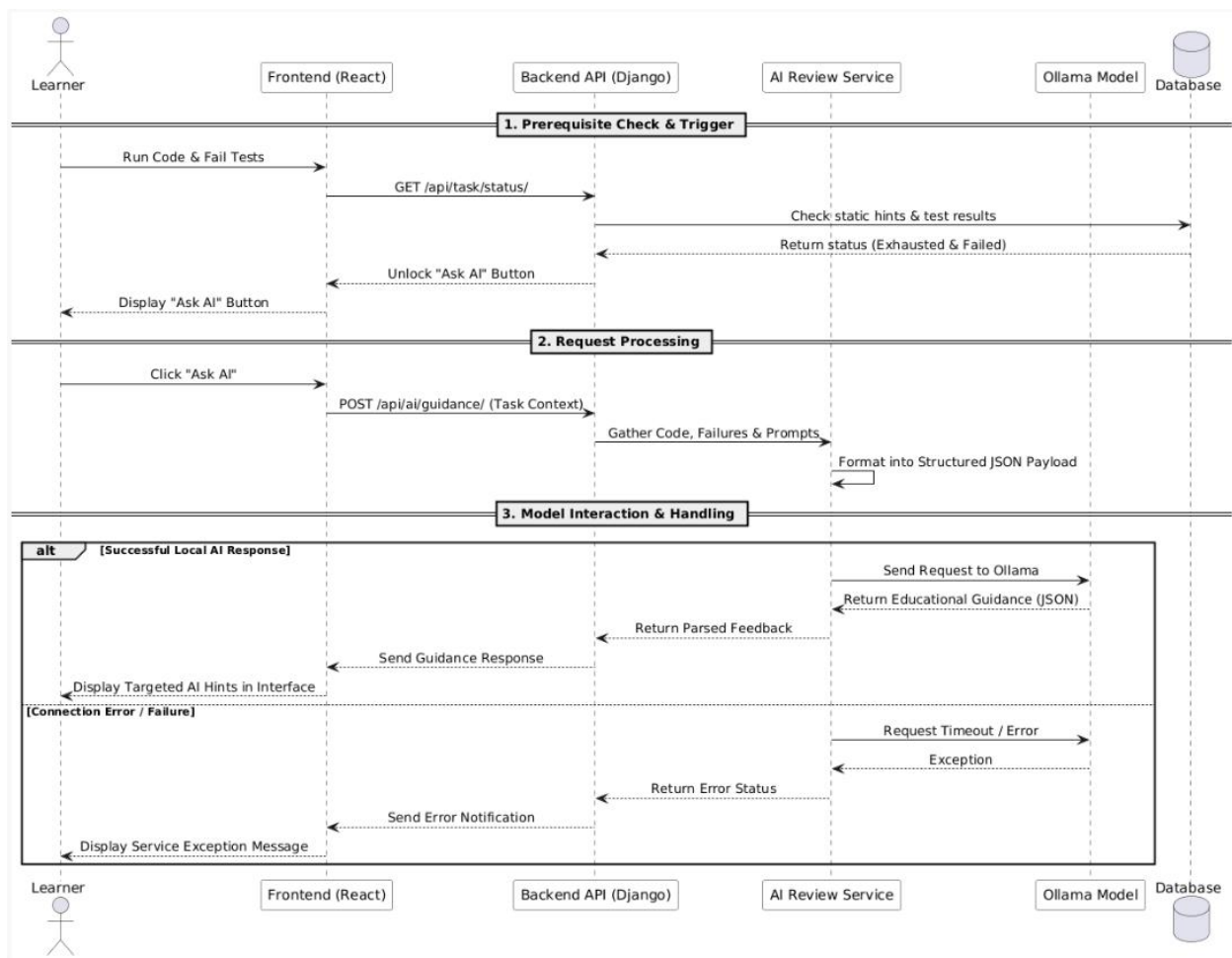
Use Case ID	UC-03
Use Case Name	AI Guidance and Code Review
Actors	Learner (Student), AI Service (Ollama Model)
Description	Allows the learner to request intelligent, educational feedback from an AI assistant after exhausting all static hints and failing code test cases, without revealing the direct solution.
Pre-conditions	1. The learner must have opened all static hints for the current task/project.

	2. The learner must have executed their code and failed the automated test cases. 3. The backend AI review structure (Ai review) and local Ollama client must be active and configured.
Main Flow	1. The learner writes code in the workspace and runs tests. 2. Upon test failure and after exhausting static hints, the system unlocks and displays the "Ask AI" button. 3. The learner clicks the "Ask AI" button. 4. The system collects the current code context, test failures, and pedagogical prompt templates, formatting them into a structured JSON payload. 5. The backend sends the request to the Ollama local AI model. 6. The AI analyzes the code against test expectations and generates targeted, educational guidance (guiding hints/debugging directions without direct answers). 7. The system receives the JSON response and displays the guidance dynamically in the learner's interface.
Alternative Flow	[A1]: Prerequisites Not Met: If the learner has remaining static hints or has not failed a test run, the "Ask AI" button remains locked/hidden, preventing premature access to AI assistance. [A2]: Model Connection Error: If the local AI service fails to respond, the system gracefully catches the exception and displays an informative error message to the learner.
Post-conditions	The AI feedback is rendered in the user interface, helping the learner debug their code independently while maintaining learning integrity.

AI Guidance System _ Activity diagram:



AI Guidance System _ Sequence diagram:

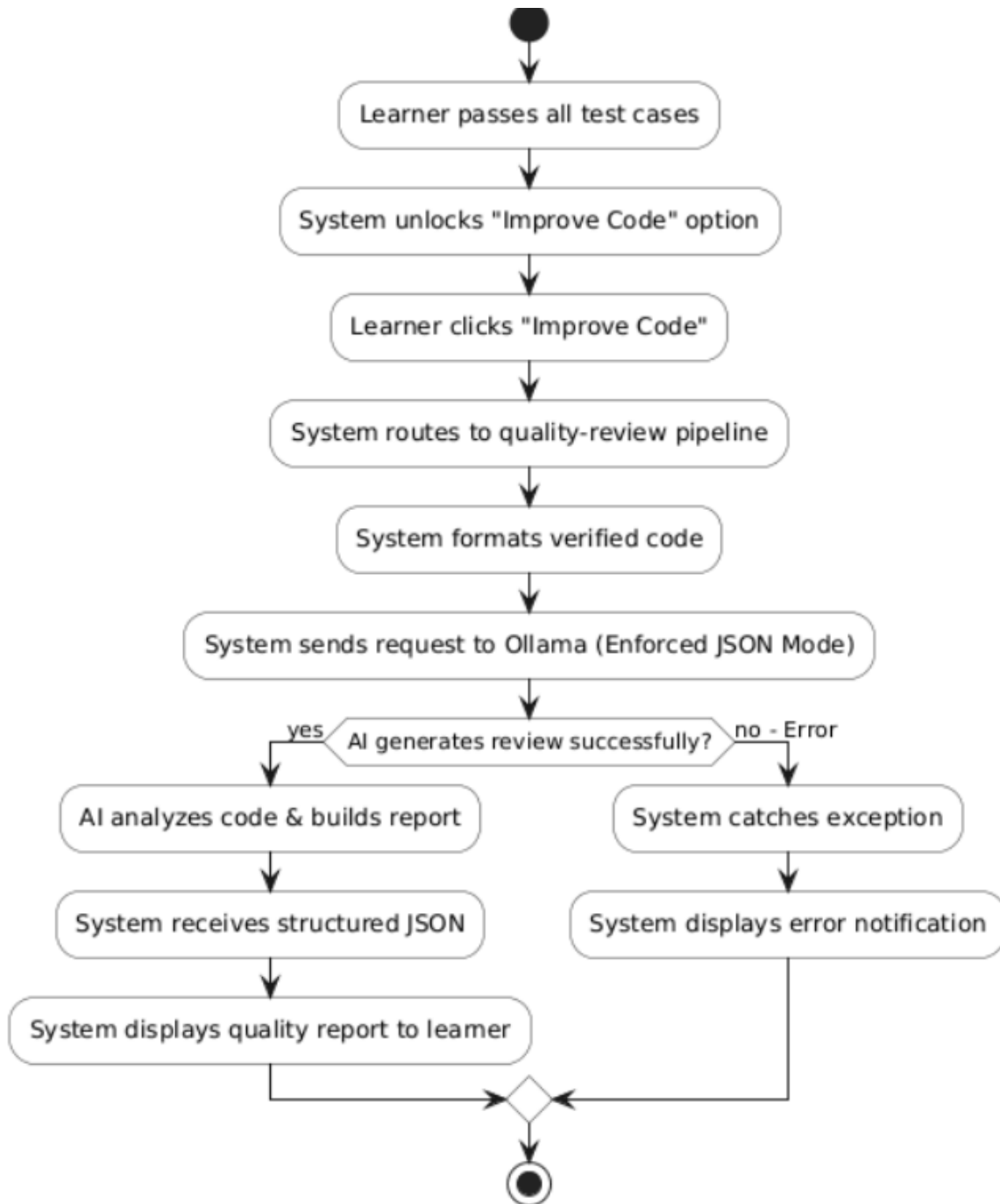


AI Code Improvement & Quality Review (UC-04):

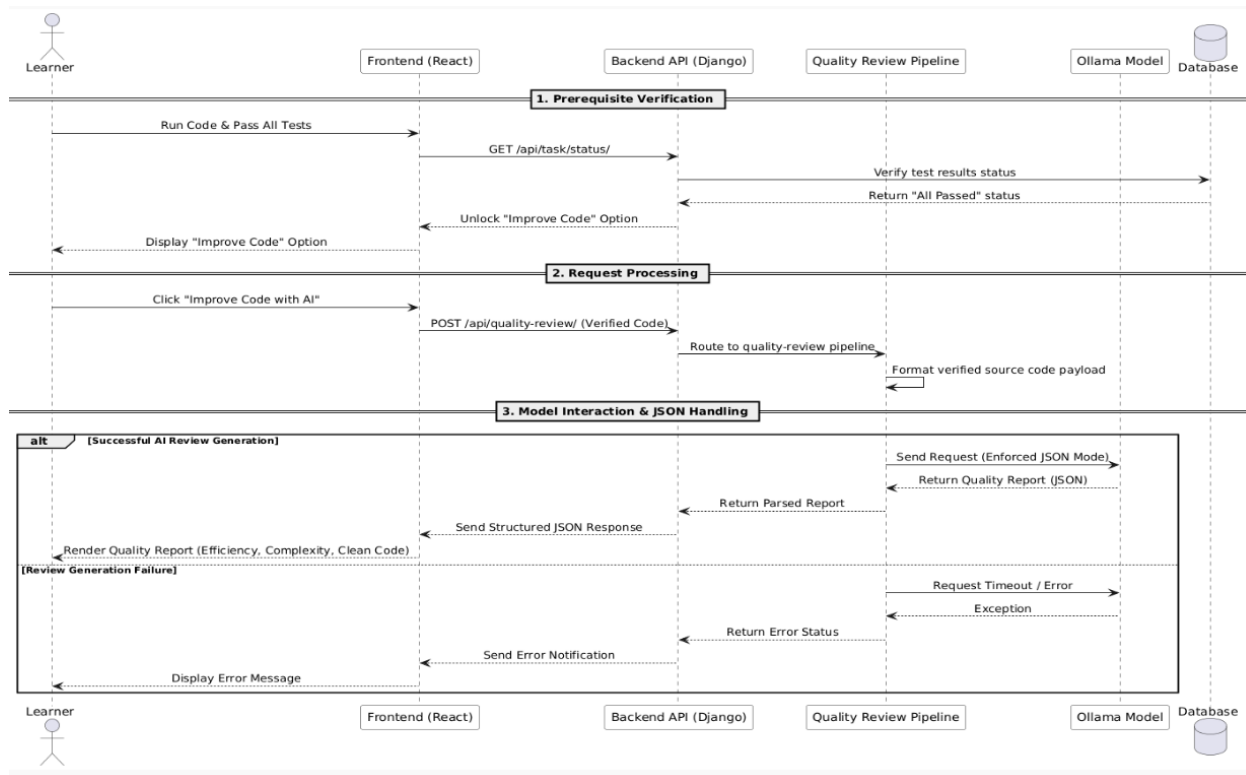
Use Case ID	UC-04
Use Case Name	AI Code Improvement & Quality Review
Actors	Learner (Student), AI Quality Review Service
Description	Allows the learner to request an automated quality assessment and improvement report after successfully passing all project test cases, providing insights on efficiency, time/space complexity, Clean Code standards, and overall performance.
Pre-conditions	1. The learner must have written code and successfully passed all automated test cases for the current task/project. 2. The dedicated quality review backend route (quality-review) and Ollama JSON mode configuration must be active.

Main Flow	1. The learner runs the code and successfully passes all test cases. 2. The system unlocks and displays the "Improve Code with AI" option in the interface. 3. The learner clicks the option to request a quality review. 4. The system routes the request to the separate quality-review backend pipeline, packaging the verified source code. 5. The backend sends a structured request to the local Ollama model with enforced JSON mode, prompting for code optimization analysis. 6. The AI analyzes the code and generates a comprehensive report covering efficiency, time/space complexity, Clean Code principles, and performance improvements. 7. The system receives the structured JSON response and presents the professional quality report to the learner.
Alternative Flow	[A1] Prerequisites Not Met: If the test cases have not passed or are still failing, the "Improve Code" feature remains hidden/locked. [A2] Review Generation Failure: If the AI model fails to return a valid response, the system gracefully handles the exception and notifies the learner to try again.
Post-conditions	A detailed code quality and improvement report is successfully rendered to the learner, guiding them on professional software engineering best practices.

AI Code Improvement & Quality Review_ Activity diagram:



AI Code Improvement & Quality Review_ Sequence Diagram :

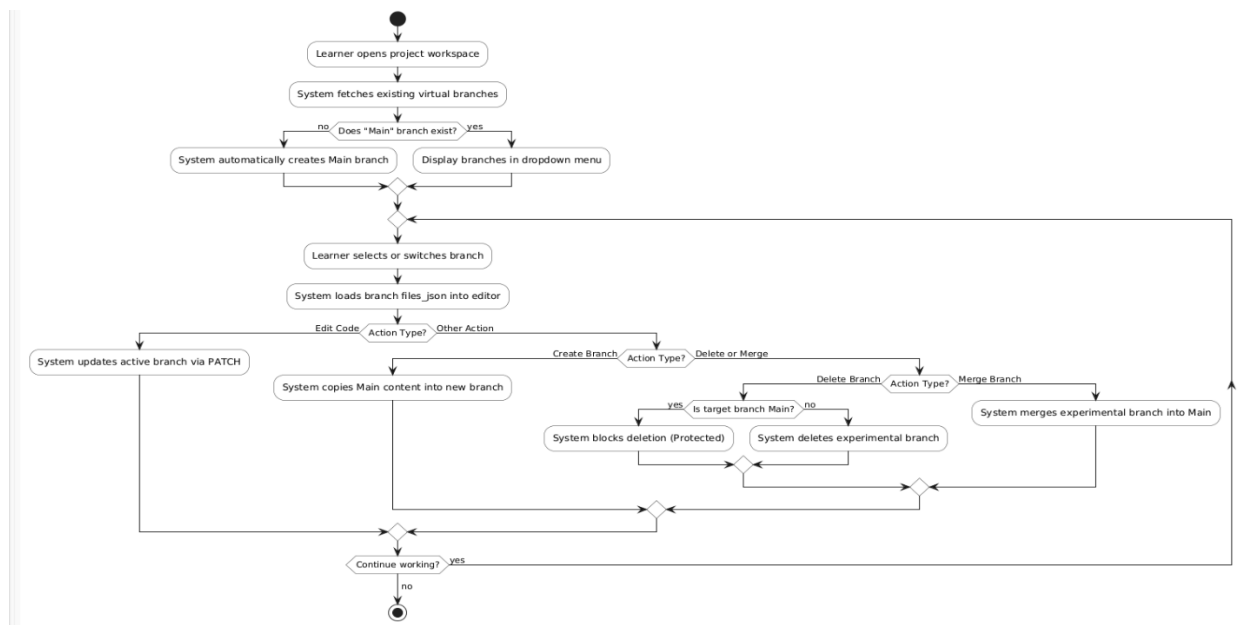


Virtual Branches Management (UC-05):

Use Case ID	UC-05
Use Case Name	Virtual Branches Management
Actors	Learner (Student)
Description	Enables learners to create, switch, manage, and merge virtual code branches to experiment with alternative solutions without altering their original Main codebase.
Pre-conditions	1. The learner must have an active project workspace session. 2. The branch management backend endpoints and database schema must be initialized.
Main Flow	1. When the learner opens a project, the system fetches the user's existing virtual branches. 2. If no "Main" branch exists, the system automatically creates it from the current workspace code. 3. The learner selects and switches to a specific branch from the dropdown menu, triggering the system to load its corresponding files Json into the editor.

	- As the learner modifies code, save/autosave operations update the active branch via PATCH requests. - The learner can create an experimental branch (which copies the Main branch content) or delete an experimental branch (while the Main branch remains protected). - Upon request, the learner can merge an experimental branch back into the Main branch.
Alternative Flow	[A1] Deleting Protected Branch: If the learner attempts to delete the "Main" branch, the system blocks the action and displays an error warning enforcing branch protection. [A2] Merge Conflict / Empty Branch: If a merge operation encounters issues or invalid states, the system notifies the learner to resolve the discrepancies before proceeding.
Post-conditions	Branch changes, experimental versions, or merged codes are safely updated and synchronized within the database and workspace interface.

Virtual Branches Management_ Activity diagram:



4.5 Modeling According to the Selected Methodology :

Since the project follows the Incremental Development Methodology, the system requirements were divided into feature lists according to development increments. Each increment includes a group of related functional requirements that can be developed, tested, and delivered as a working part of the system.

Increment 1: Core System Features:

The first increment focuses on the essential features required to make the system usable. It includes account management, learning path management, basic project management, and the learner dashboard.

Included features:

- A. Account creation.
- B. Login.
- C. Logout
- D. profile management
- E. Creating, Editing, Deleting, and Viewing learning path.
- F. Joining learning paths and viewing path details.
- G. Creating, editing, deleting, and viewing projects.
- H. Starting a project.
- I. Viewing the learner dashboard.

Increment 2: Advanced Learning and Project Features:

These features enhance the learning experience and add more interaction and control within projects.

Included features:

- A. Learner level assessment when joining a learning path.
- B. Project dependencies prerequisite management.
- C. Built-in code editor with syntax highlighting.
- D. Code execution inside the platform.
- E. Auto-saving project progress.
- F. Progress bar and time statistics.

- G. Version history and rollback.
- H. Incremental hints.
- I. Project submission and manual evaluation.
- J. Viewing and filtering submitted projects.

Increment 3: Intelligent Feedback and Analytics Features:

This increment focuses on leveraging data analytics and smart algorithms to provide a personalized learning experience and enhance system performance. It transitions the platform from a management tool to an intelligent learning assistant.

Included features:

- A. Smart Recommendation Engine: Suggesting the most suitable learning paths and projects based on the learner's previous performance and skills.
- B. Automated Feedback System: Providing real-time hints and corrections during the coding/project process using predefined patterns
- C. Peer Evaluation System: Implementing a structured workflow for students to review and rate each other's project outputs.
- D. Notification and Alert System: Real-time push notifications for project deadlines, feedback availability, and progress updates.

4.6 Complete Requirements Traceability Matrix (RTM):

Req_ID	Use Case	Title	Analysis	System Design	Coding	Test Cases
RE-FR-AM-01	UC-01	Create Account	Account creation	Authentication	Implemented	TC-04 , TC-05 , TC-06 , TC-07
RE-FR-AM-02	UC-02	Login	Login process	Authentication	Implemented	TC-01 , TC-02 , TC-03
RE-FR-AM-03	UC-03	View Profile	Profile view	Profile	Implemented	TC-08
RE-FR-AM-04	UC-04	Edit Profile	Profile edits	Profile	Implemented	TC-09 , TC-10 , TC-11
RE-FR-AM-05	UC-05	Logout	Logout process	Authentication	Implemented	—
RE-FR-LPM-06	UC-06	Create Path	Create path	Learning Path	Implemented	TC-12 , TC-13 , TC-14
RE-FR-LPM-07	UC-07	Edit Path	Edit path	Learning Path	Implemented	TC-15 , TC-16
RE-FR-LPM-08	UC-08	Delete Path	Delete path	Learning Path	Implemented	TC-17 , TC-18
RE-FR-LPM-09	UC-09	View Paths	View paths	Learning Path	Implemented	TC-19
RE-FR-LPM-10	UC-10	View Path Details	Path details	Learning Path	Implemented	TC-20

RE-FR-LPM-11	UC-11	Join Path	Join path	Learning Path	Implemented	TC-21, TC-22
RE-FR-LPM-13	UC-13	Level Assessment	Assessment	Assessment	Increment 2	TC-23, TC-24

RE-FR-PM-14	UC-14	Create Project	Create project	Project	Implemented	TC-25, TC-26, TC-27
RE-FR-PM-15	UC-15	Edit Project	Edit project	Project	Implemented	TC-28, TC-29
RE-FR-PM-16	UC-16	Delete Project	Delete project	Project	Implemented	TC-30, TC-31
RE-FR-PM-17	UC-17	View Projects	View projects	Project	Implemented	TC-32
RE-FR-PM-18	UC-18	View Projects in Path	View projects	Project	Implemented	TC-32
RE-FR-PM-19	UC-19	Project Details	Details	Project	Implemented	TC-33
RE-FR-PM-20	UC-20	Start Project	Start project	Project	Implemented	TC-34, TC-35
RE-FR-PM-21	UC-21	Project Status	Status	Project	Increment 2	
RE-FR-PM-22	UC-22	Multi Stage	Stages	Project	Increment 2	
RE-FR-PM-23	UC-23	Dependencies	Dependencies	Project	Increment 2	
RE-FR-PM-24	UC-24	Filter by Level	Filtering	Project	Increment 2	

RE-FR-PM-25	UC-25	Starter Files	Files	Project	Increment 2	
RE-FR-PM-26	UC-26	Prerequisites	Prerequisites	Project	Increment 2	
RE-FR-PM-27	UC-27	Code Editor	Editor	Code Editor	Increment 2	TC-36
RE-FR-PM-28	UC-28	Run Code	Execution	Execution	Increment 2	TC-37 , TC-38
RE-FR-PM-29	UC-29	Auto Save	Saving	Project	Increment 2	TC-39
RE-FR-PM-30	UC-30	Progress Bar	Progress	Project	Increment 2	
RE-FR-PM-31	UC-31	Statistics	Stats	Statistics	Increment 2	
RE-FR-PM-32	UC-32	Version History	Versions	Versioning	Increment 2	
RE-FR-PM-33	UC-33	Submit Project	Submission	Submission	Increment 2	TC-43
RE-FR-PM-34	UC-34	Delete Tasks	Tasks	Tasks	Increment 2	
RE-FR-PM-35	UC-35	Add Tasks	Tasks	Tasks	Increment 2	
RE-FR-PM-36	UC-36	Hints	Hints	Hints	Increment 2	TC-40 , TC-41 , TC-42
RE-FR- PM-37	UC-37	Task Sidebar	UI	UI	Increment 2	

RE-FR-PM-38	UC-38	Manual Evaluation	Evaluation	Evaluation	Increment 2	TC-44
RE-FR-PM-39	UC-39	View Submitted	Evaluation	Evaluation	Increment 2	TC-45
RE-FR-PM-40	UC-40	Filter Evaluation	Evaluation	Evaluation	Increment 2	TC-45
RE-FR-PM-41	UC-41	Dashboard	Dashboard	Dashboard	Implemented	TC-47
RE-FR-PM-42	UC-42	Automated Code Testing	Code Testing	Execution Engine	Increment 3	TC-48 TC-49
RE-FR-PM-43	UC-43	Supervisor Evaluation	Project Review	Evaluation Module	Increment 3	TC-50
RE-FR-PM-44	UC-44	Notifications	Alerts & System	Notification Service	Increment 3	TC-51
RE-FR-PM-45	UC-45	Code Sandbox	Isolated Execution	Sandbox Engine	Increment 3	TC-52
RE-FR-PM-46	UC-46	Task Management	Task Breakdown	Project Module	Increment 3	
RE-FR-PM-47	UC-47	Incremental Hints	Dynamic Hints	Hints Engine	Increment 3	
RE-FR-PM-48	UC-48	Advanced Analytics	Skill Analytics	Analytics Module	Increment 3	

4.7 Initial test cases :

This section presents a set of representative test cases used to validate the main functional requirements of the Project-Based Programming Education System. The test cases cover core functions such as account management, learning paths, project management, dashboard, and advanced project-based learning features, including automated code testing, supervisor evaluation, notifications, and code sandbox execution.

Sce-01: Login functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-01	Valid login	1. Open the login page. 2. Enter valid email/username and password. 3. Click Login.	The user is authenticated and redirected to the home/dashboard page.
TC-02	Login with empty fields	1. Open the login page. 2. Leave one or more required fields empty. 3. Click Login.	Validation messages are displayed beside the empty fields.
TC-03	Login with invalid credentials	1. Open the login page. 2. Enter invalid email/username or password. 3. Click Login.	An error message is displayed indicating invalid credentials.

Sce-02: Create account functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
--------------	-----------------	------------	-----------------

TC-04	Create account successfully	1. Open the registration page. 2. Enter valid required data. 3. Click Create Account.	A new account is created successfully and the user is redirected to the system.
TC-05	Create account with existing email	1. Open the registration page. 2. Enter an email already registered. 3. Fill remaining fields and submit.	An error message appears indicating that the email already exists.
TC-06	Create account with empty fields	1. Open the registration page. 2. Leave one or more required fields empty. 3. Click Create Account.	Validation messages are displayed beside the empty fields.
TC-07	Create account with invalid data	1. Open the registration page. 2. Enter invalid data in one or more fields. 3. Submit the form.	Validation messages are displayed beside the invalid fields.

Sce-03: Profile Management functionalities

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-08	View profile successfully	1. Log in with a valid account. 2. Open the profile page.	The system displays the user profile information correctly.

TC-09	Edit profile successfully	1. Open the profile page. 2. Click Edit. 3. Modify fields with valid data. 4. Click Save.	The system saves the updates and displays a success message.
TC-10	Edit profile with empty fields	1. Open edit profile page. 2. Clear a required field. 3. Click Save.	The system prevents saving and displays a validation message.
TC-11	Edit profile with invalid data	1. Open edit profile page. 2. Enter invalid data. 3. Click Save.	The system displays an error message beside the invalid field.

Sce-04: Add Learning Path functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-12	Add learning path successfully	1. Log in as instructor. 2. Open Add Learning Path page. 3. Enter valid path data. 4. Click Save.	The learning path is created and appears in the learning paths list.
TC-13	Add learning path with empty fields	1. Open Add Learning Path page. 2. Leave required fields empty. 3. Click Save.	Validation messages are displayed beside empty fields.
TC-14	Add duplicate learning path	1. Open Add Learning Path page. 2. Enter an existing path title. 3. Click Save.	The system displays a message indicating that the path already exists.

Sce-05: Edit Learning Path functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-15	Edit learning path successfully	1. Log in as instructor. 2. Select an existing learning path. 3. Click Edit. 4. Modify valid data and save.	The learning path is updated successfully.
TC-16	Edit learning path with invalid data	1. Open edit path page. 2. Enter invalid data. 3. Click Save.	The system displays validation messages and prevents saving.

Sce-06: Delete Learning Path functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-17	Delete learning path successfully	1. Log in as instructor. 2. Open learning paths list. 3. Click Delete for a path. 4. Confirm deletion.	The learning path is deleted and removed from the list.
TC-18	Cancel learning path deletion	1. Click Delete for a learning path. 2. Cancel the confirmation message.	The learning path remains unchanged.

Sce-07: View Learning Path functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
--------------	-----------------	------------	-----------------

TC-19	View available learning paths	1. Log in as learner or instructor. 2. Open Learning Paths page.	The system displays the list of available learning paths.
TC-20	View learning path details	1. Open Learning Paths page.	The system displays path details such as title, description,
		2. Select a specific learning path.	level, and related projects.

Sce-08: Join Learning Path functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-21	Join learning path successfully	1. Log in as learner. 2. Open a learning path. 3. Click Join.	The learner is added to the selected path and the path appears in the learner dashboard.
TC-22	Join already joined path	1. Log in as learner. 2. Open a path already joined. 3. Click Join again.	The system prevents duplicate joining or displays an appropriate message.

Sce-09: Learner Level Assessment functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-23	Complete level assessment	1. Join a learning path. 2. Answer assessment questions. 3. Submit answers.	The system calculates the result and assigns a suitable learner level.

TC-24	Cancel level assessment	1. Open level assessment page. 2. Close or skip the assessment.	The system informs the learner that level assessment is required to access suitable projects.
-------	-------------------------	---	---

Sce-10: Add Project functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-25	Add project successfully	1. Log in as instructor. 2. Open a learning path. 3. Click Add Project. 4. Enter valid project details. 5. Save.	The project is created and appears inside the selected learning path.
TC-26	Add project with empty fields	1. Open Add Project page. 2. Leave required fields empty. 3. Click Save.	Validation messages are displayed beside empty fields.
TC-27	Add duplicate project	1. Open Add Project page. 2. Enter an existing project title. 3. Click Save.	The system displays a message indicating that the project already exists.

Sce-11: Edit Project functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
--------------	-----------------	------------	-----------------

TC-28	Edit project successfully	1. Log in as instructor. 2. Open project list. 3. Click Edit for a project. 4. Modify data and save.	The project information is updated successfully.
TC-29	Edit project with invalid data	1. Open Edit Project page. 2. Enter invalid data. 3. Click Save.	The system displays validation messages and prevents saving.

Sce-12: Delete Project functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-30	Delete project successfully	1. Log in as instructor. 2. Open project list. 3. Click Delete. 4. Confirm deletion.	The project is deleted and removed from the project list.
TC-31	Cancel project deletion	1. Click Delete for a project. 2. Cancel the	The project remains unchanged.
		confirmation message.	

Sce-13: View Projects functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-32	View projects in learning path	1. Open a learning path. 2. Go to projects section.	The system displays all projects related to the selected learning path.

TC-33	View project details	1. Select a project from the list.	The system displays project title, description, tasks, requirements, resources, and level.
-------	----------------------	------------------------------------	--

Sce-14: Start Project functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-34	Start project successfully	1. Log in as learner. 2. Open a joined learning path. 3. Select a project. 4. Click Start.	The project status becomes In Progress and the project workspace opens.
TC-35	Start locked project	1. Open a project with incomplete prerequisites. 2. Click Start.	The system prevents access and displays the required prerequisite project.

Sce-15: Code Editor and Execution functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-36	Open code editor	1. Start a project. 2. Open the project workspace.	The system displays the code editor and starter files.
TC-37	Run code successfully	1. Write valid code in the editor. 2. Click Run.	The system executes the code and displays the output.

TC-38	Run code with error	1. Write code containing an error. 2. Click Run.	The system displays the error message in the output panel.
TC-39	Auto save progress	1. Modify code in the editor. 2. Wait for auto-save period or leave the page.	The system saves the latest project state automatically.

Sce-16: Hints functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-40	Request first hint	1. Open project workspace. 2. Click Request Hint.	The system displays the first hint without revealing the full solution.
TC-41	Request additional hints	1. Click Request Hint multiple times.	The system displays hints gradually from general to more specific.
TC-42	No more hints available	1. Request hints after all hints are displayed.	The system informs the learner that no more hints are available.

Sce-17: Project Submission and Evaluation:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-43	Submit project successfully	1. Complete project tasks. 2. Click Submit.	The system submits the project and changes its status to Submitted.

TC-44	Manual evaluation by instructor	1. Log in as instructor. 2. Open submitted projects. 3. Add feedback and rating. 4. Save evaluation.	The evaluation is saved and visible to the learner.
TC-45	Filter submitted projects	1. Open submitted projects list. 2. Apply evaluation status filter.	The system displays projects according to their evaluation status.

Sce-18: Dashboard functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-46	View learner dashboard	1. Log in as learner. 2. Open Dashboard.	The system displays enrolled paths, active projects, completed projects, and progress.
TC-47	Update dashboard after progress	1. Start or complete a project. 2. Return to Dashboard.	The dashboard statistics are updated according to the latest progress.

Sce-19: Automated Code Testing functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-48	Run automated tests successfully	1. Open project workspace in code editor. 2. Write code that passes criteria. 3. Click "Run Tests".	The system executes unit tests and displays a success message with passed test cases.
TC-49	Run automated tests with failures	1. Open project workspace.	The system executes tests and highlights

		2. Write code with logical/syntax errors. 3. Click "Run Tests".	the specific failing test assertions.
--	--	--	---------------------------------------

Sce-20: Supervisor Evaluation functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-50	Submit review and grade by instructor	1. Log in as instructor. 2. Open a submitted project. 3. Add comments and a grade. 4. Click Save	The evaluation is saved, and the grade becomes visible to the learner.

Sce-21: Notifications functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-51	Receive deadline or evaluation notification	1. Trigger a system event 2. Check the learner's notification panel.	The system displays a real-time notification with relevant details.

Sce-22: Isolated Code Sandbox functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result
TC-52	Secure code execution in sandbox	1. Execute resource-heavy or external code. 2. Trigger execution.	The sandbox isolates the process, handles timeouts/security rules, and returns the output safely.

4.8 Summary:

This chapter has comprehensively detailed the system requirements, architectural mapping, and rigorous validation processes for the **Buildly** platform. Following an incremental development methodology, the system

analysis transitioned from core user management and learning path structures (Increment 1) to advanced project development tools and interactive features (Increment 2), culminating in intelligent execution and evaluation modules (Increment 3).

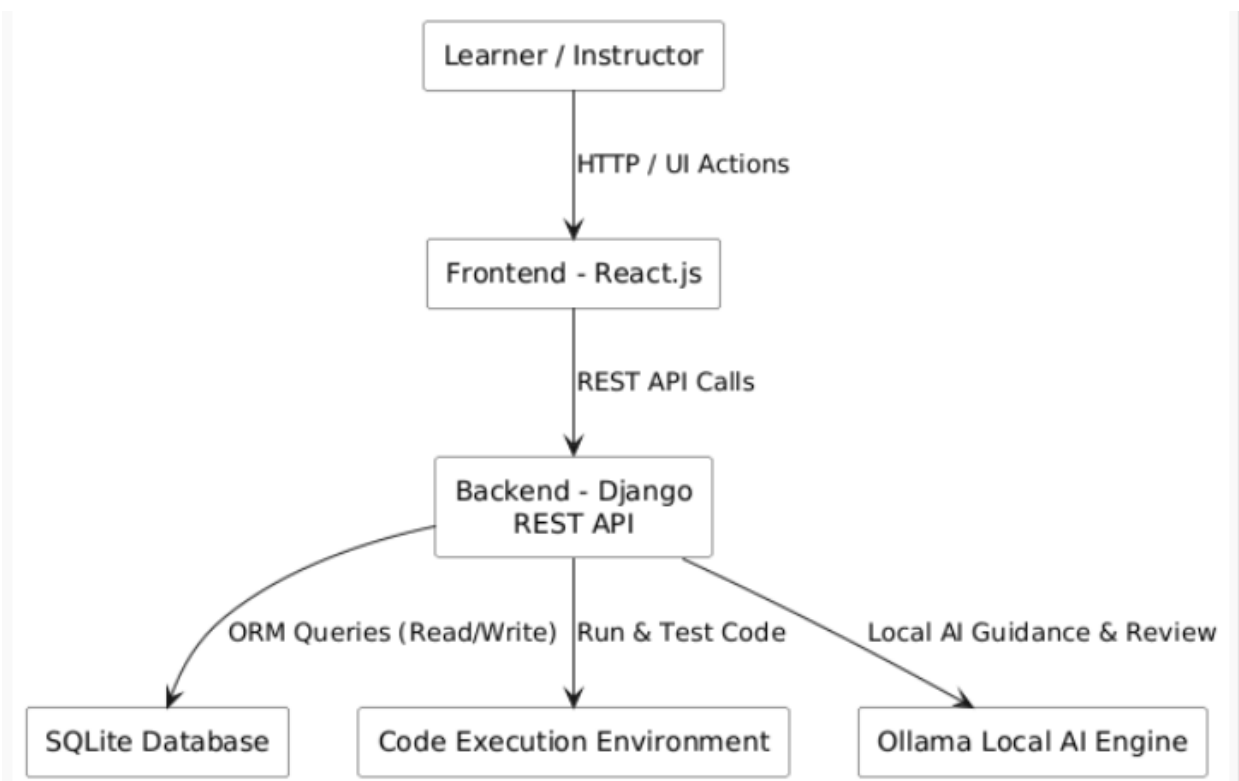
Section three: Design and architecture

Chapter 5: System architectural design

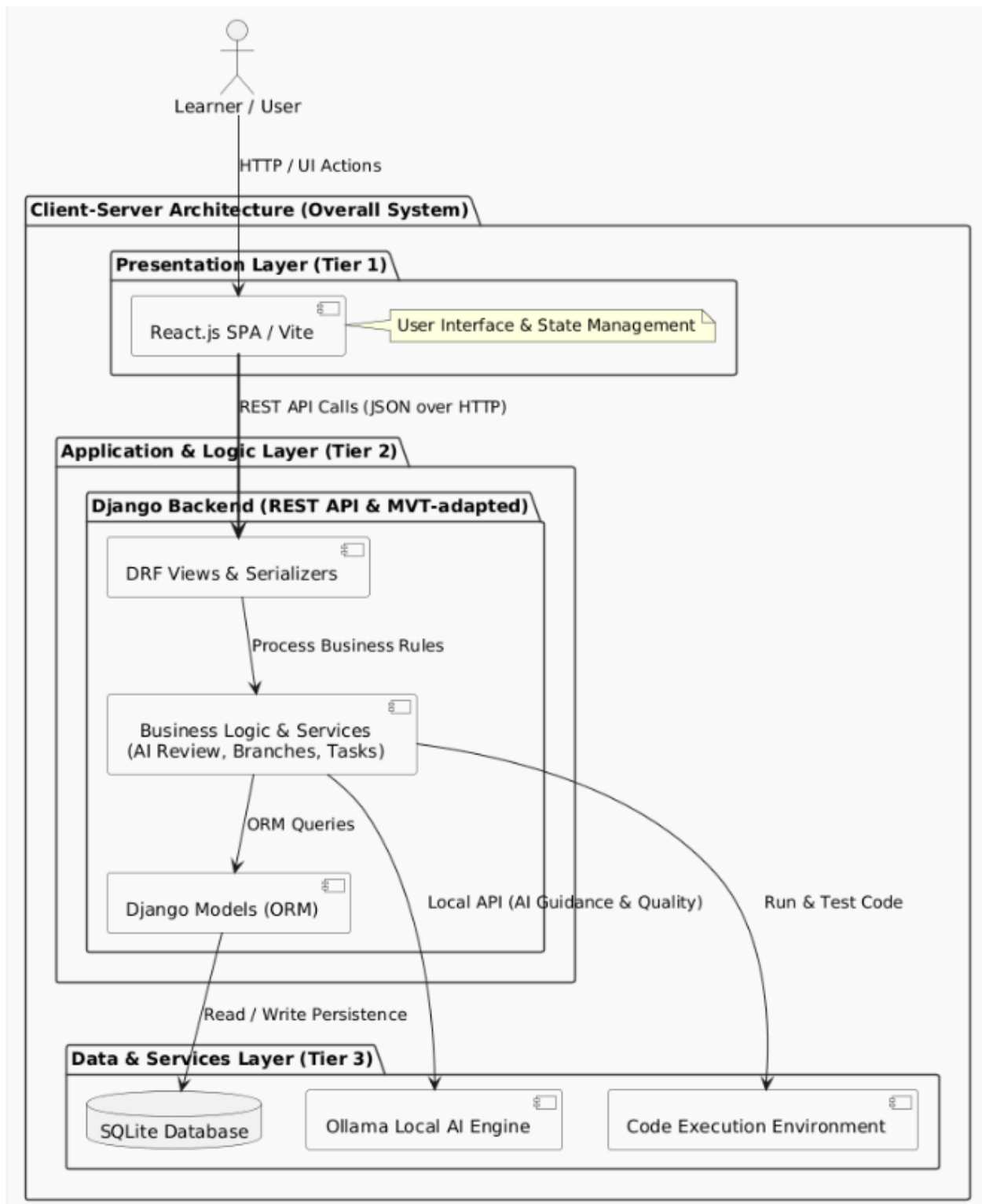
5.1 High-Level Design:

The architectural block diagram shows the main components of the system and how they interact with each other. The system follows a client-server architecture, where the frontend communicates with the backend through APIs, and the backend interacts with the database.

5.1.1 System Block Diagram:



5.1.2 Architectural design style used:



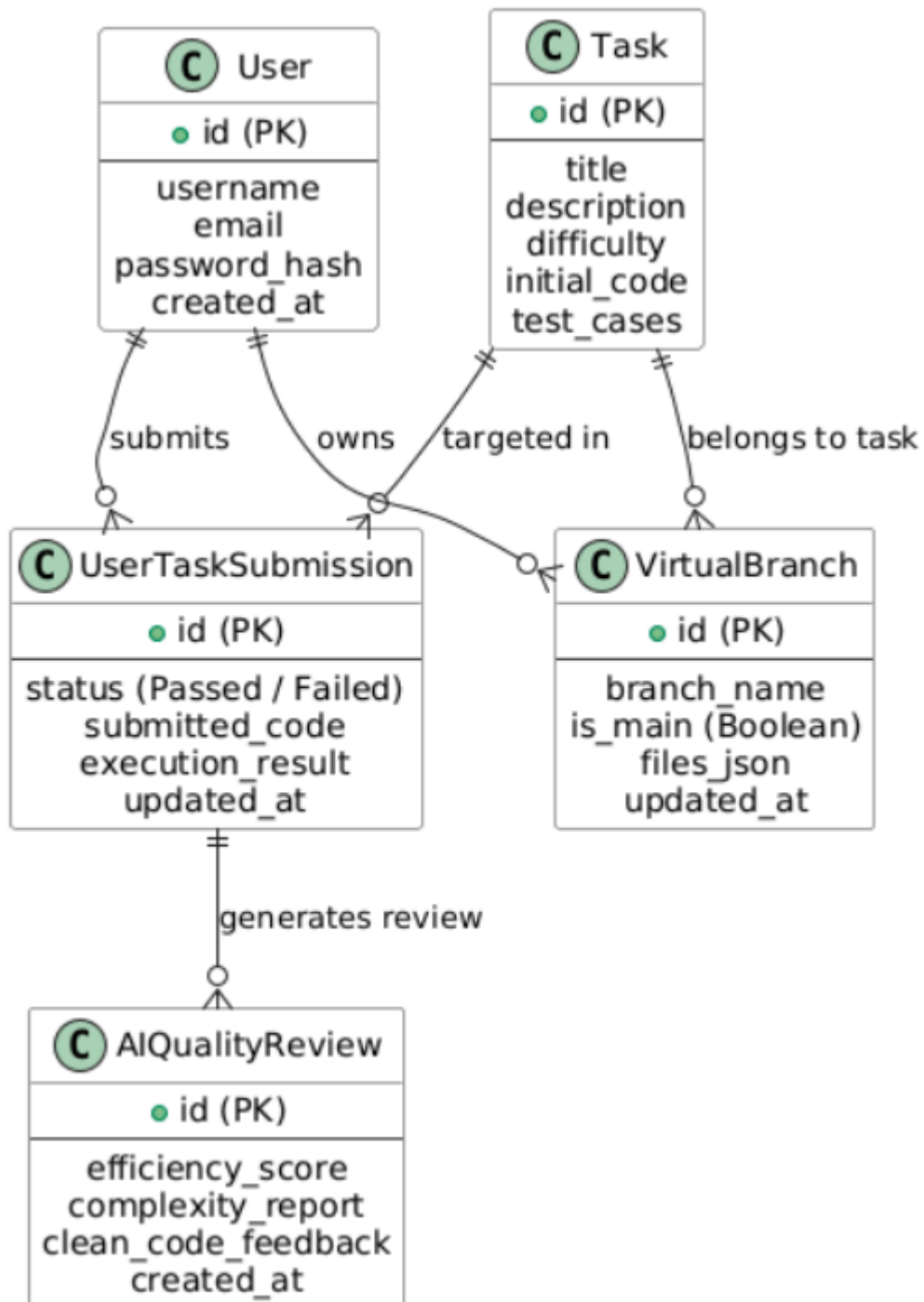
5.1.3 Architectural Pattern & System Architecture Design:

The system adopts a robust, multi-layered architectural style to ensure high cohesion, low coupling, and clear separation of concerns:

1. **Client–Server Architecture:** The system is fundamentally structured as a Client-Server model, where the frontend client (**React.js**) operates independently from the backend server (**Django REST API**), communicating seamlessly via asynchronous JSON-based HTTP requests.
2. **3-Tier Layered Architecture:**
 - **Presentation Layer (Tier 1):** Implemented via the React Single Page Application (SPA), responsible for rendering the code editor, workspaces, branch dropdowns, and AI review panels.
 - **Application Layer (Tier 2):** Hosted on the Django backend, managing business logic, RESTful endpoints, serializers, branch control systems, and coordination with local AI pipelines.
 - **Data & Services Layer (Tier 3):** Consisting of the **SQLite Database** for persistent relational storage (via Django ORM), alongside the **Ollama AI Engine** and the **Code Execution Environment** as external integration services.
3. **Django Internal MVT Adaptation:** At the server level, Django organizes its internal codebase following its native architectural principles. However, since the user interface is fully decoupled and handled by React, Django's template rendering is bypassed in favor of serving pure JSON responses, effectively adapting the Model-View-Template (MVT) pattern into a powerful Model-View-Serializer (API-driven) architecture.

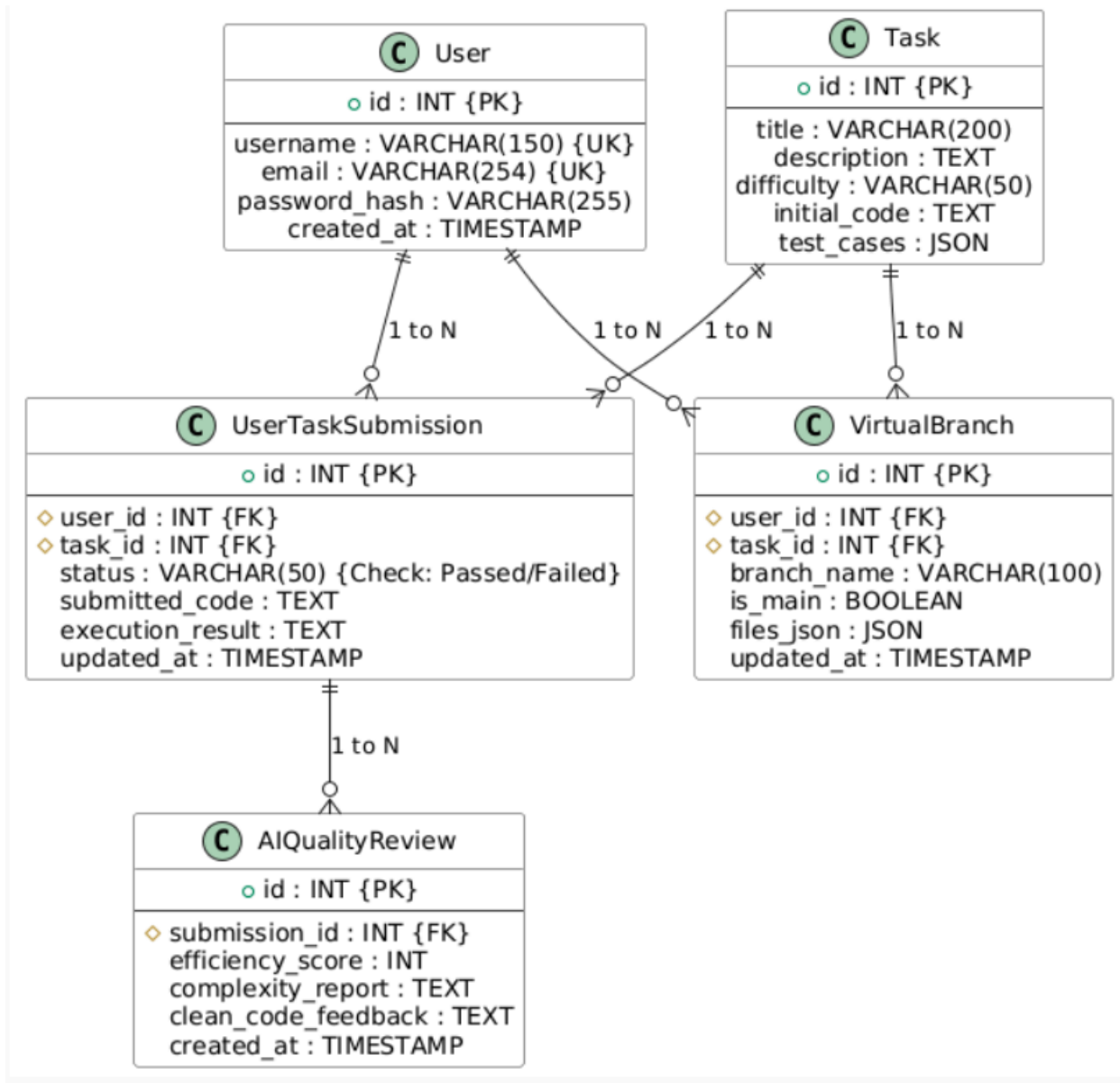
5.2 Database Design:

5.2.1 ER Diagram:



The ERD represents the main entities of the system and their relationships. It shows how users interact with tasks, code submissions, virtual branches, and AI quality reviews. This model helps organize the database structure and supports the main system functionalities.

5.2.2 Relational Schema:



The relational schema represents the database tables, fields, data types, primary keys, and foreign keys. It shows how the main entities are stored and connected in the database to support users, tasks, code submissions, virtual branches, and AI quality reviews. This model ensures data integrity and supports the core functionalities of the system.

5.2.3 Constraints and Keys:

Database constraints are used to ensure data integrity and consistency within the system. The database uses several types of keys and constraints such as primary keys, foreign keys, unique constraints, and validation checks.

Constraint Type	Example	Description
Primary Key	Id	Used to uniquely identify each record in the table.
Foreign Key	user_id, task_id, submission_id	Used to create relationships between tables and maintain referential integrity.
Unique Constraint	email, username	Prevents duplicate entries for unique user details.
Check Constraint	status IN ('Passed', 'Failed')	Ensures valid values are stored in the database.

5.3 External APIs Design:

The system uses RESTful APIs to enable communication between the frontend and backend. APIs are responsible for handling authentication, task management, code execution, virtual branches, and AI quality reviews.

Each API endpoint uses HTTP methods such as GET and POST to send and receive data in JSON format, along with standard HTTP status codes for error handling.

API Endpoint	Method	Parameters (Request)	Response
/Api/auth/login/	POST	username, password	JWT Token, Success Message
/Api/tasks/submit/	POST	Task_id, submitted code	Execution Result, Status (Passed/Failed)
/Api/branches/	GET / POST	Task_id, branch name, files Json	Branch Details / List

/Api/ai/review/	POST	submission, code	Efficiency Score, Clean Code Feedback
/Api/dashboard/	GET	None (Auth Token required)	User Progress, Completed Tasks Data

5.4 Summary:

This chapter detailed the architectural and database design of the system. It presented the system architecture, layered design approach, ER diagram, relational schema, constraints, and API design. These designs provide a clear structure that supports system scalability, maintainability, and efficient data management.

Section four: Implementation and testing

Chapter 6: Execution environment and tools

6.1 Tools Used :

- **Backend:** Django
- **Frontend:** React.js
- **Database:** SQL lite
- **Version Control:** Git & GitHub
- **Code Editor:** Visual Studio Code
- **API Testing:** Postman
- **Browser:** Google Chrome

6.2 Development Environment and Project Structure:

The system was developed in a web-based environment using Django for the backend and React for the frontend.

The project follows a modular structure divided into:

- Backend (business logic and database)
- Frontend (user interface)

6.3 Programming Languages and Frameworks:

- Programming Languages: Python, JavaScript, HTML, CSS
- Frameworks: Django (Backend), React.js (Frontend)

6.4 Database Server and Operating System:

- Database Server: SQL lite
- Operating System: Windows

6.5 Dependencies:

- Backend: Django, Django REST Framework
- Frontend: React.js, Axios
- Others: JSON, REST APIs

6.6 Version Control and Source Code Management:

- Version Control System: Git
- Platform: GitHub

6.7 Continuous Integration and Deployment (CI/CD):

CI/CD tools were not applied in this project. The development, testing, and deployment processes were performed manually.

6.8 Testing Environment:

The system was tested in a local development environment using a web browser. Manual testing was performed to verify system functionality and user interactions.

6.9 Summary:

This chapter introduced the development environment, tools, programming languages, frameworks, database server, dependencies, and testing environment used in the project. The selected technologies provided a suitable environment for implementing and testing the proposed system efficiently.

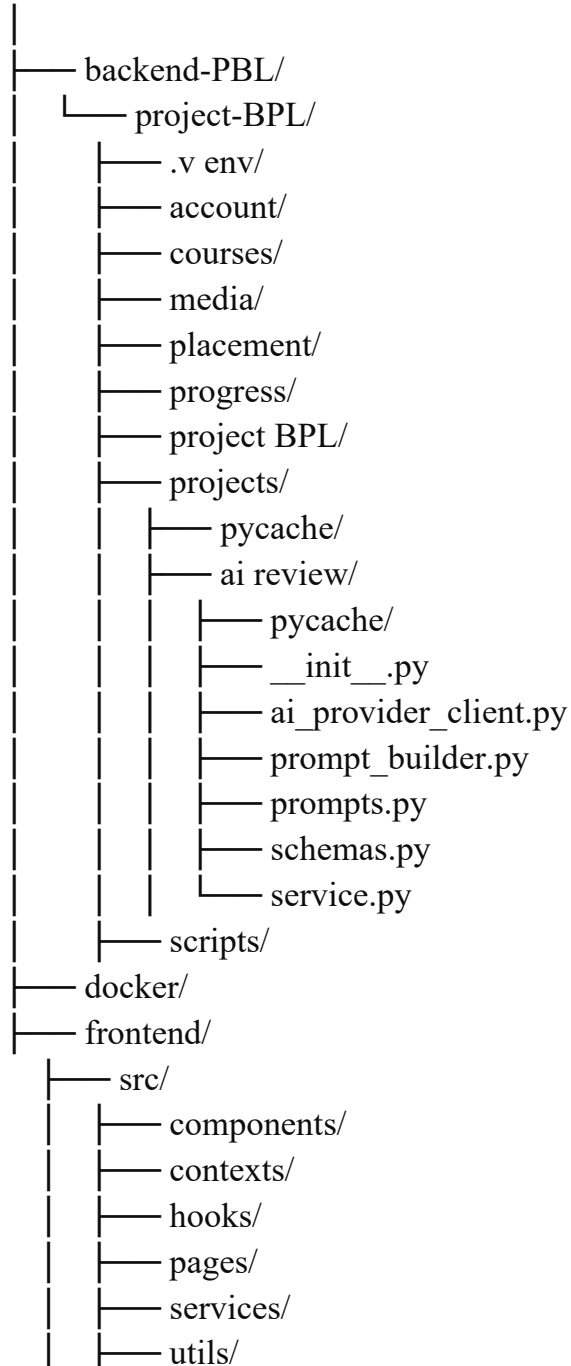
Chapter 7: Programmatic implementation

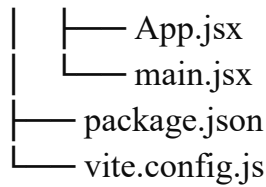
7.1 Project Structure:

The project repository is structured into two main parts: the backend server (**backend**) built with Django, and the frontend application (**frontend**) built with React.js and Vite.

BUILDLY-PROJECT-BASED-LEARNING/

"buildly" ---project-Based-learning/





7.2 Modules Implementation :

The system is divided into several independent yet interconnected backend and frontend modules to manage different aspects of the learning platform:

1. Account and Authentication Module (account):
 - Responsible for handling user registration, login, and profile management. It utilizes Django authentication alongside JWT tokens to securely manage user sessions and verify credentials.
2. Courses and Projects Module (courses & projects):
 - Implements the core logic of the project-based learning platform. It manages task creation, coding challenges, requirements, and links tasks to user workspaces and submissions.
3. Progress Tracking Module (progress):
 - Monitors user advancement through tasks and projects. It calculates completion rates and updates the database whenever a user successfully passes task requirements or tests.
4. Placement and Evaluation Module (placement):
 - Assesses user skill levels upon entering the platform to recommend appropriate learning paths and tasks tailored to their technical background.
5. Frontend Services and State Management (frontend/src):
 - The React application uses modular components, contexts, and dedicated service files (services/) to handle API communication, route protection, and dynamic user interfaces for code writing and workspace management.

7.3 API Implementation:

The system uses REST APIs to enable communication between the frontend and backend. These APIs handle user authentication, data exchange, and the execution of project-based learning tasks.

Example: Execute Code API

Item	Description
API Name	Execute Code
Method	POST
URL	/Api/projects/code/execute/
Input	code, language, files, entry Filename
Output	Execution results (stdout, stderr, and exit status)

```
class ExecuteCodeView(APIView):
    permission_classes = [permissions.IsAuthenticated]

    def post(self, request):
        code = request.data.get("code")
        language = request.data.get("language", "python")
        files = request.data.get("files")
        entry_file_name = (
            request.data.get("entryFileName")
            or request.data.get("entry_file_name")
            or "main.py"
        )

        has_files = isinstance(files, list) and len(files) > 0

        if not has_files and not code:
            return Response({"error": "No code provided"}, status=400)

        frontend_languages = {'html', 'css', 'javascript', 'typescript', 'react'}
        if language in frontend_languages:
            return Response({
                "error": "لغات الفرونت إند تُنفَّذ من المتصفح مباشرة. حذث الصفحة وحاول مرة أخرى."
            }, status=400)

        if language != "python":
            return Response({
                "error": f"فقط Python غير مدعوم على السيرفر حالياً. يدعم السيرفر '{language}' تنفيذ لغة"
            }, status=400)

        try:
            if has_files:
                outcome = run_python_in_docker(
                    files=files,
                    entry_file_name=entry_file_name,
                )
            else:
                outcome = run_python_in_docker(code)
```

```

        return Response({
            "stdout": outcome["stdout"],
            "stderr": outcome["stderr"],
            "returncode": outcome["returncode"],
        })

    except ValueError as e:
        return Response({
            "error": str(e)
        }, status=400)

    except subprocess.TimeoutExpired:
        return Response({
            "error": "Execution timeout"
        }, status=400)

    except Exception as e:
        return Response({
            "error": str(e)
        }, status=500)

```

7.4 Data Access Layer Implementation:

The data access layer is implemented using Django ORM. This layer is responsible for interacting with the database without writing raw SQL queries. The system uses Django models to represent database tables, and ORM queries are used to create, retrieve, and manage data efficiently.

Example:

```

class WorkspaceBranchCreateSerializer(serializers.Serializer):
    name = serializers.CharField(max_length=100, trim_whitespace=True)

    def validate_name(self, value):
        name = (value or '').strip()
        if not name:
            raise serializers.ValidationError(_('Branch name is required.'))
        return name

```

"WorkspaceBranchCreateSerializer" for validating and processing workspace branch creation data.

Using Django ORM makes database operations easier, safer, and more maintainable. It also helps reduce direct dependency on SQL queries and improves code readability.

7.5 Business Logic Layer:

The business logic layer contains the rules and conditions that control how the system behaves. In this project, the business logic is implemented mainly inside views and serializers.

Example: Execute CodeView:

```
frontend_languages = {'html', 'css', 'javascript', 'typescript', 'react'}
if language in frontend_languages:
    return Response({
        "error": "لغات الفرونت إند تُنفَّذ من المتصفح مباشرة. حدِّث الصفحة وحاول مرة أخرى."
    }, status=400)
```

This logic checks whether the selected language is a frontend language, preventing unnecessary server-side execution.

```
if not has_files and not code:
    return Response({"error": "No code provided"}, status=400)
```

This logic ensures that either a code snippet or files are provided before proceeding with the execution process.

These rules help prevent invalid data states, enforce system constraints, and ensure that user actions follow the required business workflow.

7.6 User Interface Implementation:

The user interface was implemented using React.js. It provides interactive pages that allow users to navigate through the system easily and perform the main functions.

Login Page:

منصة التعلم

تسجيل الدخول

مرحباً بك في نظام إدارة المسارات التعليمية

البريد الإلكتروني

أدخل بريدك الإلكتروني

كلمة المرور

أدخل كلمة المرور

تسجيل الدخول

ليس لديك حساب؟ إنشاء حساب جديد

This page allows users to log in using their email and password.

Register Page:

منصة التعلم

إنشاء حساب جديد

انضم إلى نظام إدارة المسارات التعليمية

البريد الإلكتروني

أدخل بريدك الإلكتروني

كلمة المرور

كلمة المرور (8 أحرف على الأقل)

تأكيد كلمة المرور

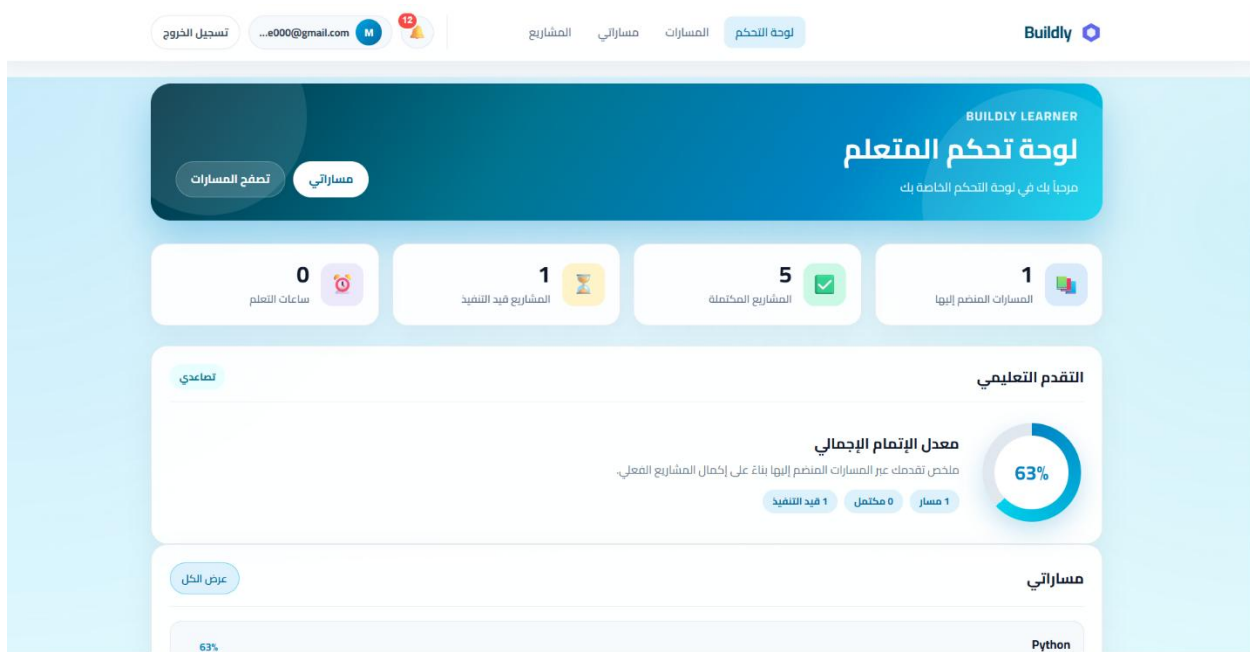
أعد إدخال كلمة المرور

إنشاء الحساب

لديك حساب بالفعل؟ تسجيل الدخول

This page allows new users to create an account as a learner or instructor.

Learner Dashboard:



This page displays the learner's enrolled paths, active projects, and progress.

Learning Paths Page:

[تسجيل الخروج](#) [...he34@gmail.com](#) [M](#) [1](#)

[المشاريع](#) [مساراتي](#) [المسارات](#) [لوحة التحكم](#) [Buildly](#)

[العودة لتفاصيل المسار](#)

00:17 [هياكل البيانات](#) السؤال 1 من 12

0% [التقدم](#)

ما ناتج `get("b", 0).{a: 1}`؟

None

0

Error

1

[Next](#)

This page shows the available learning paths that learners can browse and join.

Project Details Page:

[تسجيل الخروج](#) [...he34@gmail.com](#) [M](#) [1](#)

[المشاريع](#) [مساراتي](#) [المسارات](#) [لوحة التحكم](#) [Buildly](#)

[العودة للمشاريع](#)

مفضلة [📌](#)

Python Basic Functions [Buildly]



مشروع مبتدى لتعلم كتابة دوال Python بسيطة. ستبنى دالة ترحب ودالة جمع رقمين.

معلومات المشروع

المسار: Python

المستوى: مبتدى

لغات المشروع: Python

الوقت المقدر: 2 ساعة

الترتيب: 1

[بدء المشروع](#)

This page displays project information such as tasks, requirements, and resources.

Instructor Dashboard / Project Management Page:

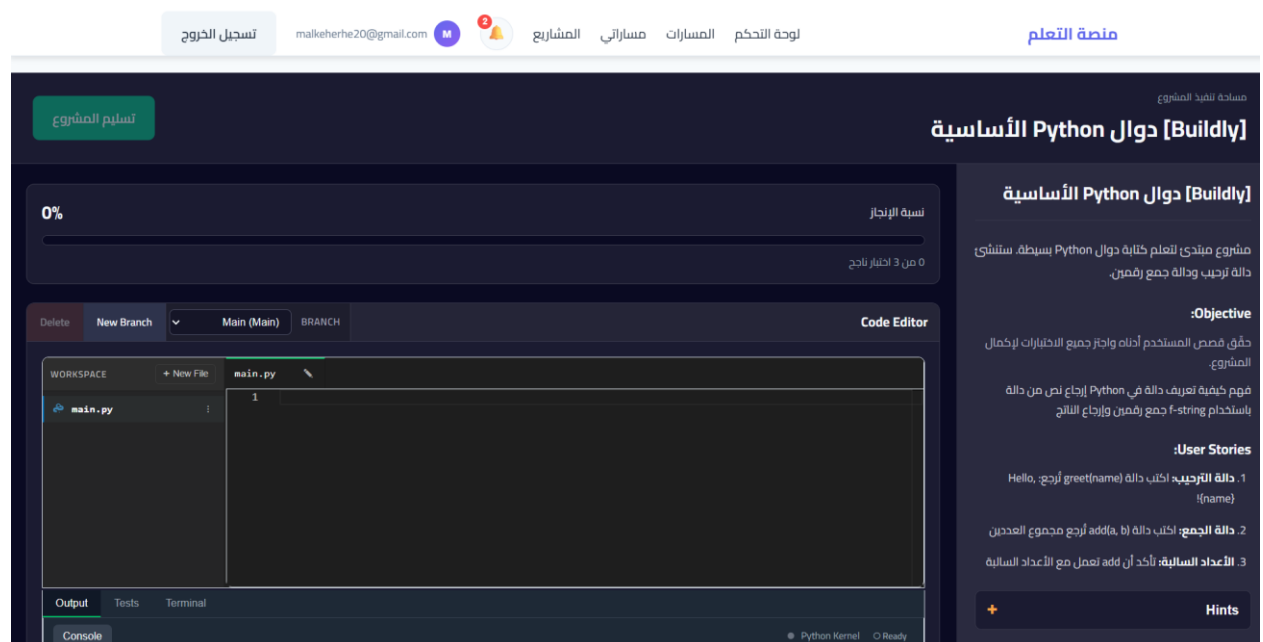
The screenshot shows the 'لوحة تحكم المشرف' (Instructor Dashboard) in Buildly Admin. The top navigation bar includes 'تسجيل الخروج' (Logout), 'Admin Omar', and a menu with 'إضافة مشروع' (Add Project), 'المشاريع' (Projects), 'إضافة مسار' (Add Course), 'المسارات' (Courses), and 'لوحة التحكم' (Dashboard). The dashboard features a purple header with 'BUILDLY ADMIN' and 'لوحة تحكم المشرف' (Instructor Dashboard) with the subtitle 'إدارة المسارات والمشاريع التعليمية' (Manage educational courses and projects). Below the header are six summary cards: '0 المفضلة' (Favorites), '1 المسارات المؤرشفة' (Archived Courses), '15 إجمالي المتعلمين' (Total Learners), '19 إجمالي المشاريع' (Total Projects), '2 المسارات النشطة' (Active Courses), and '2 إجمالي المسارات' (Total Courses). The main content area is divided into two sections: 'المسارات الحديثة' (Recent Courses) and 'الإجراءات السريعة' (Quick Actions). The 'المسارات الحديثة' section lists two courses: 'Python' (8 projects, 6 lessons) and 'Frontend Mastery' (11 projects, 11 lessons). The 'الإجراءات السريعة' section includes buttons for 'إضافة مشروع جديد' (Add New Project), 'إضافة مسار جديد' (Add New Course), 'عرض جميع المشاريع' (View All Projects), and 'عرض جميع المسارات' (View All Courses).

Placement test page:

The screenshot shows a placement test page in Buildly. The top navigation bar includes 'تسجيل الخروج' (Logout), '...he34@gmail.com', and a menu with 'المشاريع' (Projects), 'مساراتي' (My Courses), 'المسارات' (Courses), and 'لوحة التحكم' (Dashboard). The page title is 'العودة لتفاصيل المسار' (Return to Course Details). The test interface shows a progress bar at 0%, a timer at 00:17, and a question: 'ما ناتج `get("b", 0).{a: 1}`' (What is the output of `get("b", 0).{a: 1}`). The question is worth 12 points. Below the question are four radio button options: 'None', '0', 'Error', and '1'. The 'Error' option is selected. A 'Next' button is at the bottom.

This page allows students to take a test to determine their level.

Code Editor:



Delete
New Branch
Main (Main)
BRANCH
Code Editor

WORKSPACE
+ New File
main.py

main.py
1

Output
Tests
Terminal

Console
Python Kernel
Ready

مشغل الكود هنا (Enter) + (Ctrl) لفعل الكود.

Check Code

Reveal all available hints to unlock the AI Assistant.
أو استخدم (Enter) + (Ctrl) لتشغيل الكود

مشروع مبتدئ لتعلم كتابة دوال Python بسيطة. سنتشئ دالة ترحيب ودالة جمع رقمين.

Objective

تحقق فمحص المستخدم أدناه وإدراج جميع الاختيارات لإكمال المشروع.

فهم كيفية تعريف دالة في Python وإرجاع نص من دالة باستخدام f-string جمع رقمين وإرجاع الناتج

User Stories

1. دالة الترحيب: اكتب دالة greet(name) لإرجع Hello, {(name)}!

2. دالة الجمع: اكتب دالة add(a, b) لإرجع مجموع العددين

3. الأعداد السالبة: تأكد أن add تعمل مع الأعداد السالبة

+
Hints

Delete
New Branch
Main (Main)
BRANCH
Code Editor

WORKSPACE
+ New File
main.py

main.py
1
2 def greet(name):
3 return f"Hello, {name}!"
4
5 def add(a, b):
6 return a + b
7
8
9 print(add(5, 3))
10 print(add(-2, 5))
11

Output
Tests
Terminal

Console
Python Kernel
332ms
Success

مشغل الكود هنا (Enter) + (Ctrl) لفعل الكود.

STDOUT

8
3

:User Stories

- ✓ 1. **دالة الترحيب:** اكتب دالة `greet(name)` تُرجع: Hello, `!{name}`
- ✓ 2. **دالة الجمع:** اكتب دالة `add(a, b)` تُرجع مجموع العددين
- ✓ 3. **الأعداد السالبة:** تأكد أن `add` تعمل مع الأعداد السالبة

This page allows learners to write, run, and test their code inside the platform.

AI-powered code:

لوحة التحكم

المسارات

مساراتي

المشاريع

2

Code Editor

Python Kernel

Ready

أو زر التشغيل لعرض المخرجات هنا Enter

لتشغيل الكود **Ctrl** + **Enter** أو استخدم

Improve Code with AI

✦ تحسين الكود بالذكاء الاصطناعي

تقرير جودة بعد نجاح الاختبارات

درجة جودة الكود

78%

المُلخَص

تعدُّر إكمال تحليل الجودة الكامل حالياً. يمكنك الاعتماد على هذا التقييم المؤقت ثم إعادة المحاولة لاحقاً.

التعقيد (Complexity)

Time Complexity

تعقيد زمني تقريبي غير مؤكد: أعد المحاولة للحصول على تحليل أدق.

Space Complexity

تعقيد مساحي تقريبي غير مؤكد: أعد المحاولة للحصول على تحليل أدق.

نقاط القوة



This page allows the instructor to manage learning paths and projects.

The following screenshots show the main user interfaces implemented in the system. Each interface was designed to be simple, clear, and easy to use for both learners and instructors.

7.7 Intelligent Algorithms Implementation:

The system integrates intelligent algorithms to analyze code quality, evaluate submissions, and provide automated AI-driven feedback to students. Instead of training a local machine learning model from scratch, the system leverages

advanced language model APIs to process text (source code) and generate structural code quality reviews.

Chapter 8: Testing and evaluation

8.1 Test Plan:

The test plan defines the testing process used to ensure that the system functions correctly and meets the specified requirements.

What will be tested:

The system functionalities will be tested, including:

- User authentication and role management (login, registration).
- Learning path and project management.
- Advanced code editor and secure code execution (Docker integration).
- Virtual workspace branching and management.
- AI-driven code quality review and automated feedback.
- Dashboard and user interactions.

When testing will be performed:

Testing will be conducted during the development process after implementing each module (increment). Final system integration and acceptance testing will be performed after completing all modules.

Testing tools:

Manual testing is used as the main approach, supported by:

- Web browser (Google Chrome) for UI and responsiveness testing.
- Postman for API endpoint testing and validation.
- Django testing framework for unit and model testing.

8.2 Types of Testing:

The following types of testing were applied in this project:

Integration Testing:

Used to verify communication between frontend and backend through APIs.

System Testing:

The full system was tested to ensure all features work together correctly.

Acceptance Testing:

The system was validated based on the project requirements.

Usability Testing:

The user interface was tested to ensure ease of use.

Compatibility Testing:

The system was tested using a web browser (Google Chrome).

8.3 Test Case Diagrams:

8.3.1 Test Case Diagrams:

Test case diagrams represent the testing scenarios used to validate system functionalities. In this project, test cases are presented in tabular form, where each test case includes steps and expected results.

Sce-04: Add Learning Path functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result	Actual Result
TC-12	Add learning path successfully	1. Log in as instructor. 2. Open Add Learning Path page. 5. Enter valid path data. 6. Click Save.	The learning path is created and appears in the learning paths list.	The learning path was successfully created and appeared immediately in the system's learning paths list.

TC-13	Add learning path with empty fields	1. Open Add Learning Path page. 2. Leave required fields empty. 3. Click Save.	Validation messages are displayed beside empty fields.	Validation error messages appeared next to all required empty fields, preventing submission.
TC-14	Add duplicate learning path	4. Open Add Learning Path page. 5. Enter an existing path title. 6. Click Save.	The system displays a message indicating that the path already exists.	The system successfully displayed a warning message stating that the path title already exists.

Sce-15: Code Editor and Execution functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result	Actual Result
TC-36	Open code editor	3. Start a project. 4. Open the project workspace.	The system displays the code editor and starter files.	The project workspace loaded correctly, displaying the built-in code editor alongside the attached starter files.
TC-37	Run code successfully	3. Write valid code in the editor. 4. Click Run.	The system executes the code and displays the output.	The code executed successfully within the platform, and the correct output was rendered on the screen.
TC-38	Run code with error	3. Write code containing an error. 4. Click Run.	The system displays the error message in the output panel.	The system caught the error and displayed the detailed error message in the output panel.
TC-39	Auto save progress	3. Modify code in the editor. 4. Wait for auto-save period or leave the page.	The system saves the latest project state automatically.	The latest modifications to the code were saved automatically in the background without data loss.

Sce-16: Hints functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result	Actual Result
TC-40	Request first hint	3. Open project workspace. 4. Click Request Hint.	The system displays the first hint without revealing the full solution.	The first progressive hint was successfully displayed, guiding the learner without showing the full solution.
TC-41	Request additional hints	1. Click Request Hint multiple times.	The system displays hints gradually from general to more specific.	Subsequent hints appeared sequentially and gradually, moving from general direction to more specific steps.
TC-42	No more hints available	1. Request hints after all hints are displayed.	The system informs the learner that no more hints are available.	A notification message correctly appeared informing the learner that no further hints are available.

Sce-19: Automated Code Testing functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result	Actual Result
TC-48	Run automated tests successfully	4. Open project workspace in code editor. 5. Write code that passes criteria. 6. Click "Run Tests".	The system executes unit tests and displays a success message with passed test cases.	Unit tests were successfully executed, showing a success message and listing all passed test cases.
TC-49	Run automated tests with failures	4. Open project workspace. 5. Write code with logical/syntax errors. 6. Click "Run Tests".	The system executes tests and highlights the specific failing test assertions.	The test runner executed the tests and precisely highlighted the failing assertions and logic gaps.

Sce-20: Supervisor Evaluation functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result	Actual Result
--------------	-----------------	------------	-----------------	---------------

TC-50	Submit review and grade by instructor	5. Log in as instructor. 6. Open a submitted project. 7. Add comments and a grade. 8. Click Save	The evaluation is saved, and the grade becomes visible to the learner.	The evaluation, comments, and grade were successfully saved and updated in the learner's dashboard.
-------	---------------------------------------	---	--	---

Sce-21: Notifications functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result	Actual Result
TC-51	Receive deadline or evaluation notification	3. Trigger a system event 4. Check the learner's notification panel.	The system displays a real-time notification with relevant details.	A real-time notification with correct event details appeared instantly in the notification panel.

Sce-22: Isolated Code Sandbox functionalities:

Test Case ID	Test Case Title	Test Steps	Expected Result	Actual Result
TC-52	Secure code execution in sandbox	3. Execute resource-heavy or external code. 4. Trigger execution.	The sandbox isolates the process, handles timeouts/security rules, and returns the output safely.	The sandbox successfully isolated the execution process, managed security rules/timeouts, and returned safe output.

Additional test cases covering other system functionalities are defined but not fully presented in this report due to space limitations

8.3.2 Test Results:

The test results show that the system functions correctly for the vast majority of test cases, confirming the stability and reliability of the newly integrated features.

- Most test cases passed successfully on the first execution.
- Minor edge-case errors related to code execution formatting and API responses were identified and successfully fixed during testing.
- The system performs as expected under normal usage conditions

The condition	Num
Passed	50
Failed	2
Fixed	2

8.3.3 Test Results and Analysis:

The testing process was conducted using a set of predefined test cases to validate the system functionality and newly integrated modules.

Most test cases were successfully passed, indicating that the system works correctly under normal conditions. A small number of test cases initially failed due to minor issues such as input validation errors, API integration edge cases, and user interface inconsistencies. These issues were identified and fixed during the testing phase.

The condition	Num
Passed	50

Failed	2
Fixed	2

The results show that the system is stable and meets the main functional requirements. Core functionalities such as authentication, learning path management, advanced code execution, and AI-driven quality reviews performed as expected.

8.4 Summary:

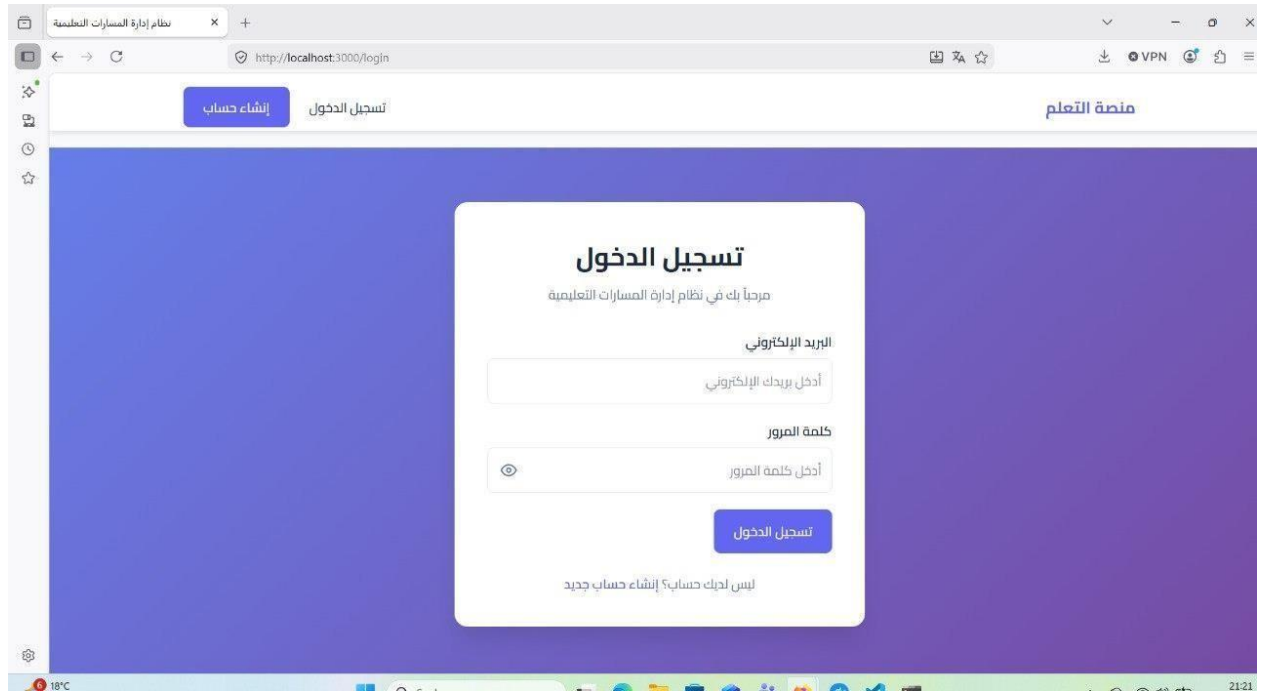
This chapter presented the comprehensive testing and evaluation process of the system, covering both core functionalities and newly integrated intelligent features. Different types of testing, structured test cases, quality metrics, and evaluation procedures were thoroughly discussed. The final results demonstrated that the system performs correctly, maintains high stability, and successfully satisfies all functional requirements and user expectations.

Section five: Results and recommendations

Chapter 9: Results and analysis

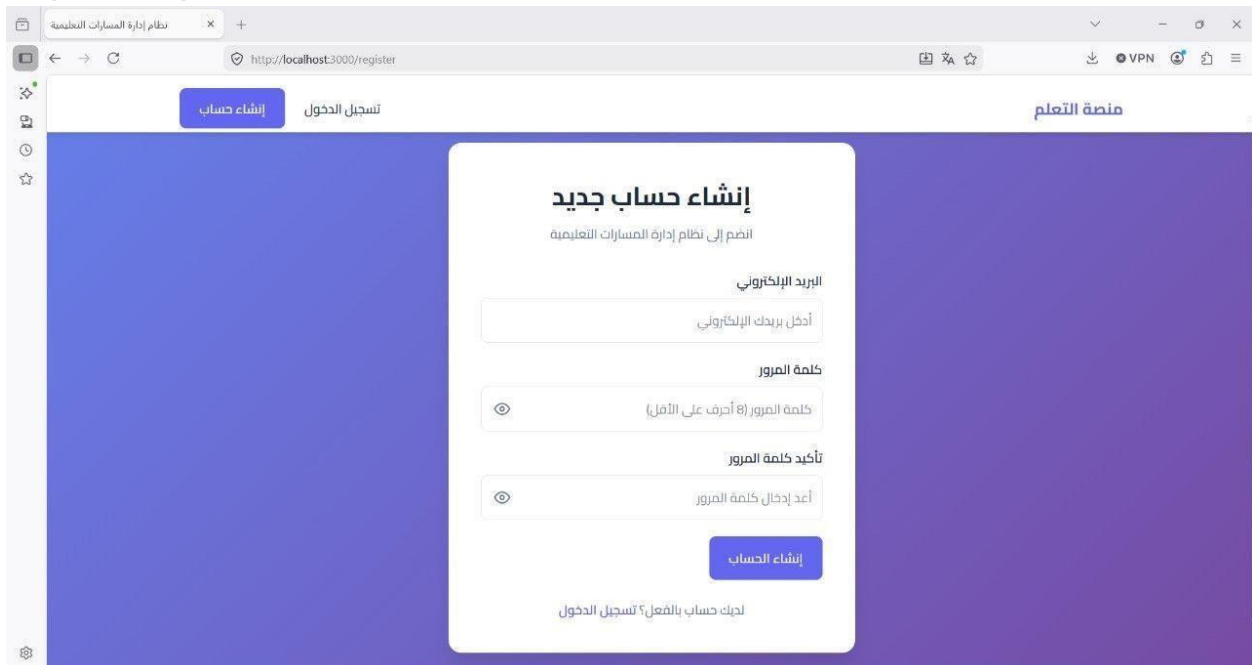
9.1 System implementation results:

Login Page:



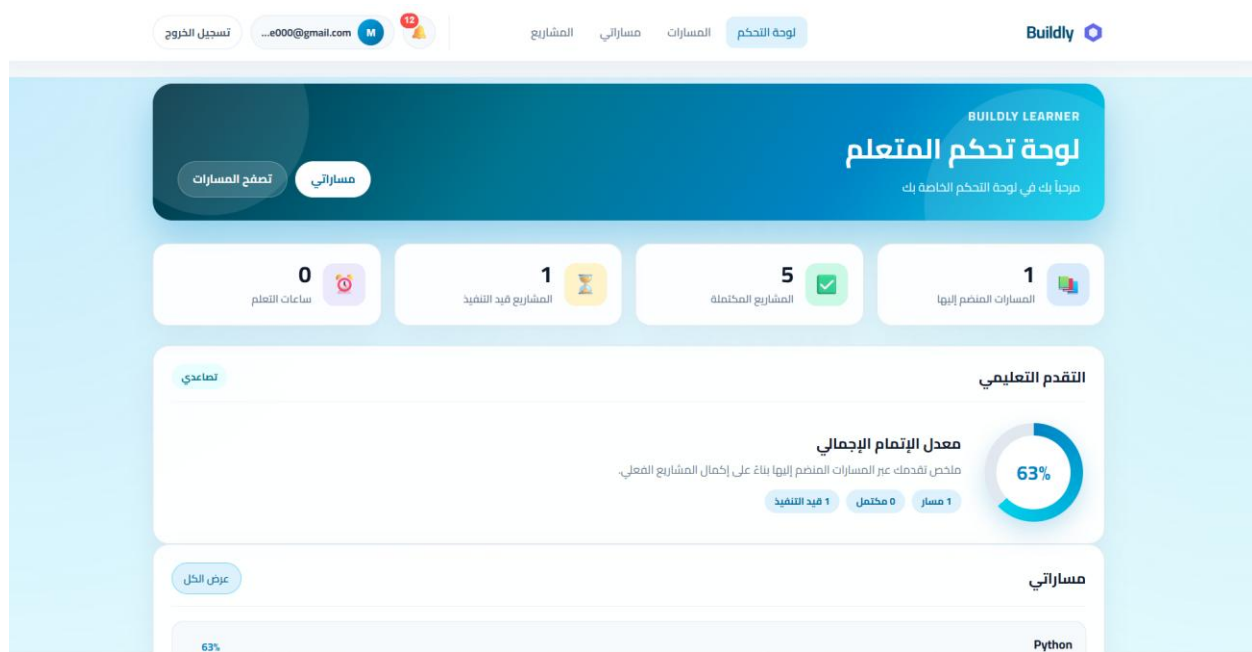
The screenshot shows a web browser window with the URL `http://localhost:3000/login`. The page has a dark blue background with a white login form in the center. The form is titled "تسجيل الدخول" (Login) and includes a subtitle "مرحباً بك في نظام إدارة المسارات التعليمية" (Welcome to the Educational Path Management System). The form contains two input fields: "البريد الإلكتروني" (Email) and "كلمة المرور" (Password). Below the password field is a "تسجيل الدخول" (Login) button. At the bottom of the form, there is a link "ليس لديك حساب؟ إنشاء حساب جديد" (Don't have an account? Create a new account). The browser's address bar shows the URL, and the page title is "منصة التعلم" (Learning Platform).

Register Page:



The screenshot shows a web browser window with the URL `http://localhost:3000/register`. The page has a dark blue background with a white registration form in the center. The form is titled "إنشاء حساب جديد" (Create New Account) and includes a subtitle "انضم إلى نظام إدارة المسارات التعليمية" (Join the Educational Path Management System). The form contains three input fields: "البريد الإلكتروني" (Email), "كلمة المرور" (Password), and "تأكيد كلمة المرور" (Confirm Password). Below the password field is a "إنشاء الحساب" (Create Account) button. At the bottom of the form, there is a link "لديك حساب بالفعل؟ تسجيل الدخول" (Already have an account? Login). The browser's address bar shows the URL, and the page title is "منصة التعلم" (Learning Platform).

Learner Dashboard:



Learning Paths Page:

تسجيل الخروج

...he34@gmail.com

M

1

المشاريع

مساراتي

المسارات

لوحة التحكم

Buildly

العودة لتفاصيل المسار

Python Placement Test

اختبار متكيف لتحديد مستواك في أساسيات Python · هياكل البيانات · OOP · تولد النظام أسئلة مبردة لكل طالب باستخدام الذكاء الاصطناعي، وتتغير صعوبتها حسب أدائك، ثم يعرض لك المشاريع المناسبة لمستواك في مسار Python.

أساسيات Python · هياكل البيانات · OOP

اختبار متكيف + AI

20 ثانية لكل سؤال

12 سؤالاً

لديك 20 ثانية فقط لكل سؤال، إذا انتهى الوقت يستبدل السؤال بسؤال جديد مباشرة دون الانتقال للسؤال التالي — لمنع البحث عن الإجابة. لا يمكن العودة للسؤال السابق بعد الإجابة.

Start Assessment

تسجيل الخروج

...he34@gmail.com

M

1

المشاريع

مساراتي

المسارات

لوحة التحكم

Buildly

العودة لتفاصيل المسار

00:17

هياكل البيانات

السؤال 1 من 12

التقدم

0%

ما ناتج `get("b", 0).a`؟

None

0

Error

1

Next

← العودة لتفاصيل المسار

تم إكمال اختبار تحديد المستوى

تم حفظ نتيجتك وانضمامك لمسار Python.

المستوى النهائي: Advanced

Ability Score: 0.4

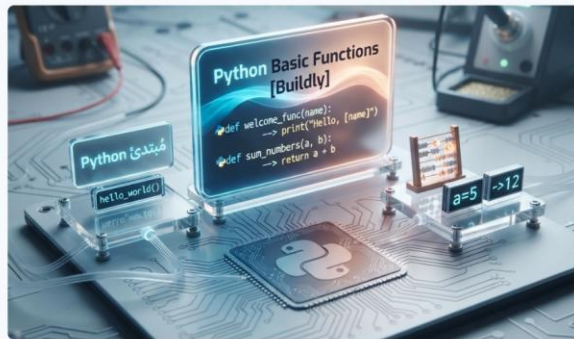
OOP هياكل البيانات PYTHON أساسيات

عرض المشاريع المناسبة لمستواك

Project Details Page:

← العودة للمشاريع

Python Basic Functions [Buildly]



الوصف

مشروع مبتدى لتعلم كتابة دوال Python بسيطة، ستتشى دالة ترحيب ودالة جمع رقمين.

معلومات المشروع

المسار: Python

المستوى: مبتدئ

لغات المشروع: Python

الوقت المقدر: 2 ساعة

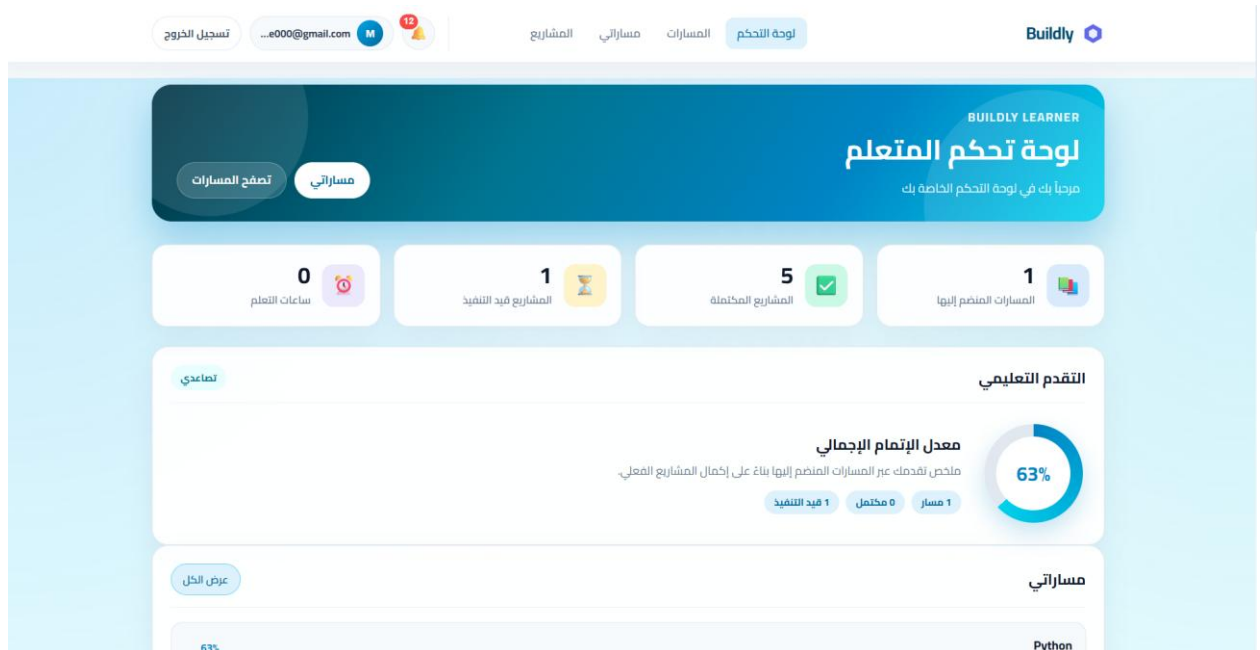
1 الترتيب:

بدء المشروع

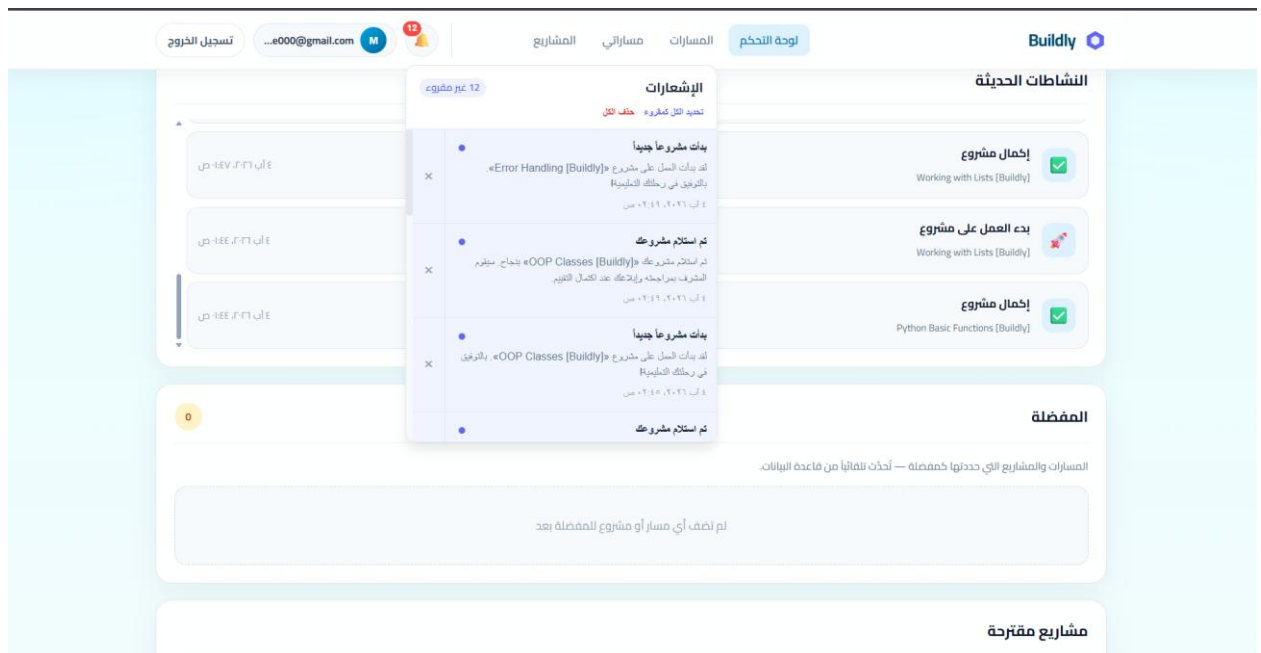
Dashboard:



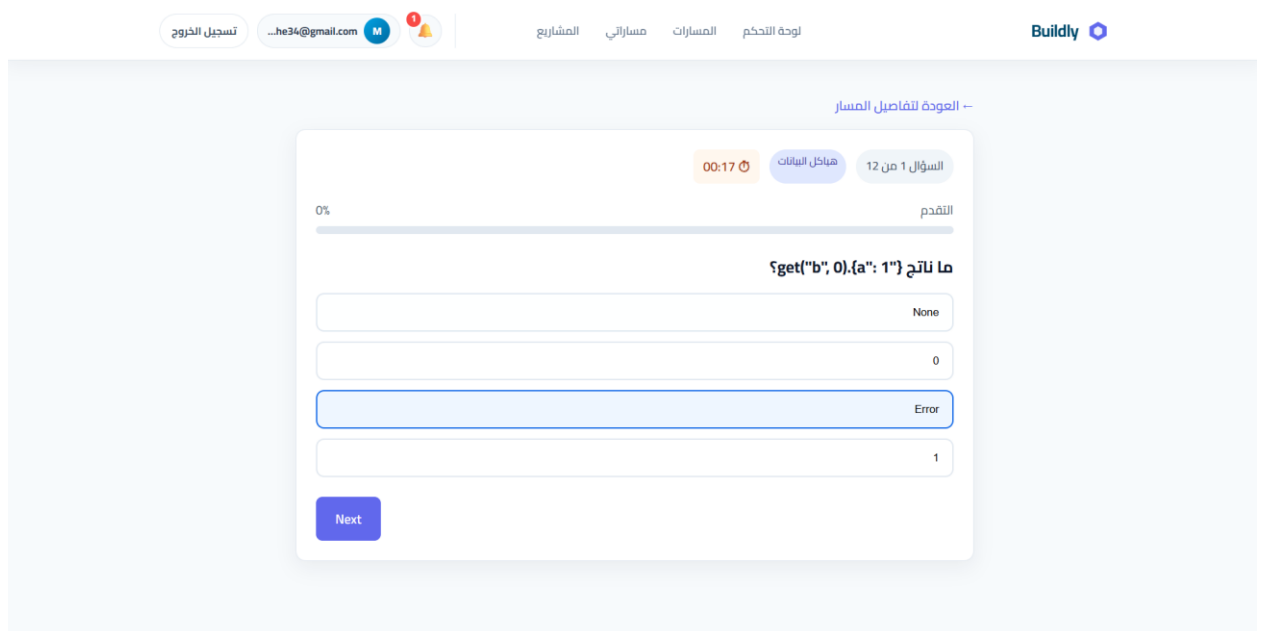
The screenshot shows the 'BUILDLY ADMIN' dashboard. At the top, there's a navigation bar with 'لوحة التحكم' (Dashboard) selected, and other links like 'المسارات', 'إضافة مسار', 'المشاريع', and 'إضافة مشروع'. The main header area includes 'BUILDLY ADMIN' and 'لوحة تحكم المشرف' (Admin Control Panel) with a subtitle 'إدارة المسارات والمشاريع التعليمية' (Manage educational paths and projects). Below this, there are six summary cards: 'المفضلة' (0), 'المؤرشفة' (1), 'المتعلمين' (15), 'إجمالي المشاريع' (19), 'المسارات النشطة' (2), and 'إجمالي المسارات' (2). The main content area is divided into two sections: 'المسارات الحديثة' (New Paths) and 'الإجراءات السريعة' (Quick Actions). The 'المسارات الحديثة' section lists two paths: 'Python' (8 projects, 6 lessons) and 'Frontend Mastery' (11 projects, 11 lessons). The 'الإجراءات السريعة' section has four buttons: 'إضافة مشروع جديد' (Add New Project), 'إضافة مسار جديد' (Add New Path), 'عرض جميع المشاريع' (View All Projects), and 'عرض جميع المسارات' (View All Paths).



The screenshot shows the 'BUILDLY LEARNER' dashboard. The navigation bar includes 'لوحة التحكم' (Dashboard) selected, and other links like 'المسارات', 'مساراتي', and 'المشاريع'. The main header area includes 'BUILDLY LEARNER' and 'لوحة تحكم المتعلم' (Learner Control Panel) with a subtitle 'مرحباً بك في لوحة التحكم الخاصة بك' (Welcome to your personal control panel). Below this, there are four summary cards: 'ساعات التعلم' (0), 'المشاريع قيد التنفيذ' (1), 'المشاريع المكتملة' (5), and 'المسارات المنضم إليها' (1). The main content area is divided into two sections: 'التقدم التعليمي' (Educational Progress) and 'مساراتي' (My Paths). The 'التقدم التعليمي' section shows a 'معدل الإتمام الإجمالي' (Overall Completion Rate) of 63%, with a breakdown: 1 completed project, 0 in progress, and 1 pending. The 'مساراتي' section shows a progress bar for 'Python' at 63%.



Placement test page:



Code Editor:

منصة التعلم

لوحة التحكممساراتمساراتيالمشارك
malkeherhe20@gmail.comتسجيل الخروج

مساعدة تنفيذ المشروع

تسليم المشروع

0%

نسبة الإنجاز

0 من 3 اختبار ناجح

DeleteNew BranchMain (Main)BRANCHCode Editor

WORKSPACE+ New Filemain.py

main.py1

OutputTestsTerminalConsolePython KernelReady

Objective

مشروع مبتدئ لتعلم كتابة دوال Python بسيطة، ستشتمل دالة ترحيب ودالة جمع رقمين.

User Stories

1. دالة الترحيب: اكتب دالة greet(name) لترجع: Hello, {name}!

2. دالة الجمع: اكتب دالة add(a, b) لترجع مجموع العددين

3. الأعداد السالبة: تأكد أن add تعمل مع الأعداد السالبة

Hints

DeleteNew BranchMain (Main)BRANCHCode Editor

WORKSPACE+ New Filemain.py

main.py1

OutputTestsTerminalConsolePython KernelReady

مشروع مبتدئ لتعلم كتابة دوال Python بسيطة، ستشتمل دالة ترحيب ودالة جمع رقمين.

Objective

مشروع مبتدئ لتعلم كتابة دوال Python بسيطة، ستشتمل دالة ترحيب ودالة جمع رقمين.

User Stories

1. دالة الترحيب: اكتب دالة greet(name) لترجع: Hello, {name}!

2. دالة الجمع: اكتب دالة add(a, b) لترجع مجموع العددين

3. الأعداد السالبة: تأكد أن add تعمل مع الأعداد السالبة

Hints

Check Code

Reveal all available hints to unlock the AI Assistant. أو استخدم `Ctrl + Enter` لتشغيل الكود

أو زر التشغيل لعرض المخرجات هنا `(Ctrl + Enter)` لتشغيل الكود.

The screenshot shows a code editor interface. At the top, there are tabs for 'Delete', 'New Branch', and 'Main (Main)'. The main editor area displays a Python file named 'main.py' with the following code:

```
1
2 def greet(name):
3     return f"Hello, {name}!"
4
5 def add(a, b):
6     return a + b
7
8
9 print(add(5, 3))
10 print(add(-2, 5))
11
```

Below the code editor, there is an 'Output' section with a 'Console' tab. It shows the output of the code execution:

```
STDOUT
8
3
```

The console also indicates 'Python Kernel 332ms' and 'Success'.

- ### :User Stories
- ✓ 1. دالة الترحيب: اكتب دالة `greet(name)` تُرجع: Hello, {name}
 - ✓ 2. دالة الجمع: اكتب دالة `add(a, b)` تُرجع مجموع العددين
 - ✓ 3. الأعداد السالبة: تأكد أن `add` تعمل مع الأعداد السالبة

AI-powered code:

2

المشاريع

مساراتي

المسارات

لوحة التحكم

Code Editor

Python Kernel

Ready

أو زر التشغيل لعرض المخرجات هنا

Improve Code with AI

لتشغيل الكود **Ctrl** + **Enter** أو استخدم

تحسين الكود بالذكاء الاصطناعي

تقرير جودة بعد نجاح الاختبارات

درجة جودة الكود

78%

الملخص

تعدّز إكمال تحليل الجودة الكامل حالياً. يمكنك الاعتماد على هذا التقييم المؤقت ثم إعادة المحاولة لاحقاً.

التعقيد (Complexity)

Time Complexity

تعقيد زمني تقريبي غير مؤكد؛ أعد المحاولة للحصول على تحليل أدق.

Space Complexity

تعقيد مساحي تقريبي غير مؤكد؛ أعد المحاولة للحصول على تحليل أدق.

نقاط القوة

نقاط القوة

- الحل اجتاز الاختبارات بنجاح، وهذا أساس جيد لمراجعة الجودة.

نصائح الكود النظيف

- راجع أسماء المتغيرات والدوال وهل تعبر بوضوح عن الغرض.
- قسّم المنطق الطويل إلى وحدات أصغر يسهل فهمها وصيانتها.

تحسينات الأداء

- ابحث عن التكرار غير الضروري في الحلقات أو العمليات المكلفة.
- فكر هل يمكن تقليل العمل المكرر دون تغيير صحة النتيجة.

9.2 Comparative Analysis with Existing Systems:

This section presents a comprehensive comparison between the proposed system and existing programming learning platforms, taking into account recent enhancements such as AI-driven code reviews and virtual workspace branching.

Feature	Proposed System	Existing Systems
Speed	Fast response (1–3 seconds) including code execution and AI analysis	Moderate response time
Accuracy & Intelligence	High accuracy combined with automated AI code quality reviews	Varies depending on platform (often basic syntax check)

Workspace Flexibility	Supports virtual workspace branching and sandbox experiments	Mostly static project environments
Cost	Free (academic platform designed for educational purposes)	Most advanced platforms require high subscription fees

The proposed system provides highly competitive performance compared to existing systems. By integrating advanced features like automated AI feedback and isolated virtual environments alongside fast execution and cost-effectiveness, it serves as a robust and superior educational tool for programming learners.

9.3 Achievement of Objectives:

The main objectives defined in Chapter 1 were successfully achieved through the comprehensive implementation of the proposed system and its advanced modules.

- The system provides a robust project-based learning environment, which helps learners apply theoretical knowledge in practice.
- A complete platform was developed including authentication, learning paths, and project management.
- The system supports interactive learning through advanced features such as the secure code editor and real-time code execution.
- Virtual workspace branching and automated AI-driven code quality reviews were successfully integrated to enhance the learning process.
- User progress tracking and dashboard functionalities were successfully implemented.

Overall, the system meets its intended goals and provides an effective, intelligent, and comprehensive solution for enhancing programming education.

9.4 Strengths and Weaknesses Analysis:

➤ Strengths

- The system provides a project-based learning approach, which enhances practical skills compared to traditional learning methods.

- It integrates multiple advanced features such as learning paths, project management, virtual workspace branching, and secure code execution in one unified platform.
- It incorporates automated AI-driven code quality reviews to provide instant, intelligent feedback to learners.
- The system offers a user-friendly and responsive interface, making it easy for learners to navigate and use.
- It supports interactive learning, allowing users to write and execute code directly inside the system.

➤ **Weaknesses :**

- The system lacks performance optimization under extreme heavy load, as it was tested primarily under normal concurrent conditions.
- Testing was heavily reliant on manual approaches, with limited automated unit testing scripts.
- The platform is currently web-based only and does not provide native mobile applications.

Despite these limitations, the system provides a solid, scalable foundation that can be further improved and extended in future work.

9.5 Summary:

This chapter presented the system implementation results and evaluated the proposed platform through comparative analysis, achievement of objectives, strengths and weaknesses analysis, and key performance indicators. The results demonstrated that the system successfully provides an effective, intelligent, and practical solution for project-based programming education, integrating advanced features like automated AI reviews and virtual workspace environments.

Chapter 10: Conclusion and Recommendations

10.1 Project Summary and Main Achievements:

This project presented an advanced Project-Based Programming Education System designed to enhance learning by seamlessly combining theoretical knowledge with practical application. The system provides structured learning paths, real-world projects, and an interactive environment that supports learners in developing their programming skills effectively.

The main achievements of this project include the successful implementation of a robust web-based platform featuring user authentication, learning path management, project management, and an advanced integrated code editor. Furthermore, the system successfully integrated cutting-edge capabilities such as virtual workspace branching and automated AI-driven code quality reviews. Combined with real-time code execution and comprehensive progress tracking, the platform delivers a highly practical, intelligent, and engaging learning experience.

10.2 Challenges and Difficulties:

During the development and expansion of this project, several challenges were encountered:

Technical Challenges:

- Difficulties were faced while integrating the frontend (React) with the backend (Django), especially in handling complex APIs, asynchronous data exchange, and state management.
- Implementing advanced features such as secure code execution environments, workspace branching, and integrating automated AI-driven code quality review APIs required significant technical research and effort.

Time Constraints:

- The project and its new intelligent modules had to be completed within a strict academic timeframe, which required careful task prioritization, agile iterations, and precise time management.

Organizational Challenges:

- Managing multiple interconnected modules and maintaining system consistency across the core platform and new advanced features required strong structural organization and coordination.

Despite these challenges, they were successfully and gradually resolved through continuous testing, rigorous debugging, learning, and effective planning.

10.3 Future Work:

Several improvements can be made to enhance the system in the future:

- Expanding AI-based features to include personalized project recommendations and fully adaptive learning paths based on student performance.
- Further enhancing the code editor with advanced collaborative tools, such as real-time multi-user coding and intelligent auto-completion.
- Expanding the platform's execution engine to support a wider range of programming languages and frameworks.
- Developing native mobile applications for iOS and Android to increase platform accessibility and user engagement.
- Implementing automated unit testing pipelines and advanced performance monitoring metrics under heavy concurrent load.

10.4 Recommendations:

Based on the development, implementation, and evaluation of this project, the following recommendations are suggested:

- It is strongly recommended to adopt project-based learning approaches combined with interactive tools in programming education to significantly improve practical and problem-solving skills.
- Educational institutions and academic departments can benefit from integrating similar intelligent platforms to enhance student engagement, streamline code evaluations, and improve overall learning outcomes.
- Future developers and researchers should focus on improving system scalability, incorporating advanced AI automation, and handling heavy concurrent loads.
- Continuous testing, rigorous quality assurance, and active user feedback collection should be constantly prioritized to ensure continuous system improvement and usability.

10.5 Summary:

This chapter concluded the project by summarizing the main achievements, discussing the encountered technical and organizational challenges, and

presenting valuable recommendations and future improvements. The proposed intelligent system successfully achieved its educational and technical objectives, integrating advanced features such as AI code reviews and virtual workspaces to provide a strong and scalable foundation for future development and enhancement.

Section Six: Appendices and References

Section A: Requirements Gathering & Gap Analysis

A.1 Appendix Introduction:

Due to the nature of the **Buildly** project as a project-based learning (PBL) programming platform integrating artificial intelligence, traditional student questionnaires were replaced by an "**Observational Gap Analysis**" of student workflows and a comprehensive comparative study of existing educational platforms. This appendix documents the synthesized findings and insights that directly drove the system's functional and non-functional requirements.

1. Summary of Student Workflow Observations:

By closely observing learners and identifying real-world programming hurdles, the following core requirements were derived:

- **Task Fragmentation & Tracking Challenges:** Students struggle to manage and track multi-stage projects, justifying the implementation of progress bars and sidebar task tracking.
- **The Feedback Delay Gap:** Waiting for manual reviews stalls self-learning, validating the integration of an AI-driven guidance system that provides incremental hints without spoiling solutions.
- **Need for Safe Experimentation:** Learners require isolated spaces to test alternative code approaches without breaking their stable code, leading to the Virtual Branches feature.

2. Platforms Comparative Analysis Summary:

Benchmarking against existing platforms (Codecademy, FreeCodeCamp, SoloLearn, The Odin Project) highlighted critical industry gaps:

- Most platforms lack an embedded, fully functional browser IDE capable of multi-file execution and real-time project building.
- A lack of adaptive evaluation frameworks that dynamically tailor difficulty to the student's actual performance level, a limitation successfully solved in our platform via the adaptive assessment module.

Section B: Detailed Design

B.1 Class & Component Diagrams:

B.1.1 Detailed Class Diagram and Relationships:

The object-oriented architecture of the **Buildly** platform is structured to support project-based learning (PBL), version control emulation, and AI-driven interactions. The core classes and their relationships include:

- **User (Abstract Base Class):** Represents system users with common attributes (userID, username, email, password Hash).
- **Student (Inherits from User):** Manages student-specific attributes like skill level and learning progress.
- **Instructor (Inherits from User):** Manages course content, project definitions, and administrative oversight.
- **Project:** Represents a practical programming project containing multiple tasks (1 to * aggregation relationship with Task).
- **Task:** Represents individual milestones within a project, tracking completion status and associated code requirements.
- **Virtual Branch:** Emulates version control features, allowing students to test code alternatives without breaking their main stable branch (1 to * relationship with Student and Project).
- **AI Assistant:** Handles interaction logic with the AI inference engine to process student code snippets and generate progressive hints.

B.1.2 Component Diagram:

The system is partitioned into independent, loosely-coupled architectural modules:

- **Presentation Tier (Frontend):** Developed using React.js to provide a responsive user interface and an embedded code editor.
- **Application Tier (Backend):** Built using Django / REST APIs to handle business logic, user authentication, and routing requests.
- **AI Processing Tier:** Manages communication with the AI models for automated code analysis, error detection, and hint generation.
- **Data Storage Tier:** Utilizes a relational database management system to persistently store user data, project definitions, progress tracks, and audit logs.

B.2 Modules Design:

The platform architecture is divided into four primary subsystems:

- **Authentication & User Management Module:** Handles secure user registration, session management, role-based access control (RBAC) for students and instructors, and profile settings.
- **PBL & Project Management Module:** Manages the delivery of project-based learning paths, tracks step-by-step student progress, and updates completion metrics.
- **Embedded IDE & Virtual Branching Module:** Provides an in-browser code editing environment coupled with version control mechanisms, allowing users to safely create, test, and merge virtual code branches.
- **AI Guidance & Evaluation Module:** Captures student code inputs, communicates with the AI engine, evaluates structural logic, and returns targeted, progressive hints.

B.3 APIs Design:

The backend exposes a robust RESTful API architecture. Key endpoints implemented in the system include:

Method	Endpoint	Description
POST	/Api/v1/auth/login	Authenticates user credentials and returns a secure JWT access token.
GET	/Api/v1/projects/{id}	Retrieves detailed metadata, description, and task lists for a specific project.
POST	/Api/v1/ai/hints	Submits the student's current code context to generate an incremental AI hint.
POST	/Api/v1/virtual-branch/save	Saves intermediate code states within the student's virtual branch environment.
GET	/Api/v1/student/progress	Fetches progress analytics, completed tasks, and performance metrics for the active user.

B.4 Smart Algorithms Design:

B.4.1 Data Flow Diagrams (DFD) & State Machine Diagrams:

- **Data Flow Diagram (DFD):** Illustrates the flow of data across the system. Input data (such as source code submissions or hint requests originating from the student interface) flows through the Django backend API controller, gets processed by the AI integration

layer, queries the relational database for project parameters, and returns structured feedback to the frontend dashboard.

- **State Machine Diagram (Task Lifecycle):** Tracks the lifecycle states of a student project task: Not Started → In Progress → Under AI Review → Completed (or redirected back to In Progress if corrections are required).

B.4.2 Algorithmic Pseudo-code:

Below is the pseudo-code for the **Incremental Hint Generation Algorithm**, which evaluates student code and provides guidance without revealing the complete solution.

B.5 UI/UX Design & System Screenshots:

- **Wireframes & Mockups:** Initial layout blueprints were designed using wireframing tools to map out the core user journeys, focusing on minimizing cognitive load in the student dashboard and code editor layout.
- **Final System Screenshots:** The production-ready user interface features an intuitive layout containing:
 1. **The Main Dashboard:** Showcasing active project paths, overall progress percentage bars, and quick-access navigation.
 2. **The Integrated IDE View:** Featuring split panes for project instructions, the code editor interface, and the live terminal output.
 3. **The AI Assistant Panel:** Providing contextual feedback loops and incremental hints side-by-side with the working workspace.

Appendix C: Detailed Test Results

C.1 Introduction to System Testing:

This appendix documents the testing phase of the **Buildly** platform, providing comprehensive test logs, bug-tracking summaries, and snapshots of resolved errors. The testing strategy validates functional requirements, ensures code security, verifies user authentication procedures, and tests the reliability of the AI-driven hint generation engine.

C.2 Test Logs & Execution Summary:

The system underwent rigorous black-box, white-box, and integration testing. Below is a summary log of key test cases executed across different modules:

Test ID	Module / Component	Test Scenario / Input	Expected Result	Actual Result	Status
TC-01	Authentication	Valid user credentials login	Access granted & JWT token issued	Access granted & token returned	Passed
TC-02	Authentication	Invalid password submission	Error response with status code 400/401	Authentication rejected securely	Passed
TC-03	Project Management	Fetching project metadata (/Api/v1/projects/{id})	Returns project details and task list	Valid JSON response returned	Passed
TC-04	Virtual Branching	Saving intermediate code states	Code changes committed to virtual branch	Code state persisted successfully	Passed
TC-05	AI Guidance Engine	Submitting empty code snippet for hint	Code changes committed to virtual branch	Returned 400 Bad Request error	Passed

C.3 Test Snapshots & Verification Logs:

The testing phase confirmed that all major functional blocks operate correctly under normal and edge-case conditions:

- **Unit Testing Coverage:** Verified backend modules using Python's unit test / pytest frameworks, achieving high code coverage across utility functions and API views
- **UI Responsiveness Verification:** Evaluated layout adaptability across multiple screen resolutions to ensure the split-pane IDE and dashboard elements render without overlapping.

Appendix D: User Manual & Installation Guide

D.1 Introduction:

This user manual provides step-by-step instructions for installing, configuring, and operating the **Buildly** platform. It is intended for both students utilizing the project-based learning environment and instructors managing coursework and system deployment.

D.2 System Prerequisites & Environment Requirements:

Before installing the platform, ensure that your local development or server environment meets the following minimum requirements:

- **Operating System:** Cross-platform support (Windows 10/11, macOS, or Linux Ubuntu 20.04+).
- **Python Environment:** Python 3.10 or higher.
- **Node.js Environment:** Node.js (v16.x or higher) and npm for frontend asset management
- **Database:** SQLite (default for development) or PostgreSQL (recommended for production).
- **API Keys:** A valid OpenAI API key for configuring the AI-driven hint generation module.

D.3 System Installation & Setup:

Follow these steps to set up the system locally from the source repository:

Step 1: Clone the Repository

Open your terminal and clone the project repository from version control

Step 2: Backend Setup (Django & REST API)

1. Create and activate a virtual environment
2. Install the required Python dependencies
3. Configure the environment variables by creating a .env file in the root directory
4. Run database migrations and start the backend server

Step 3: Frontend Setup (React.js)

1. Open a new terminal window, navigate to the frontend directory, and install dependencies
2. Start the React development server

The frontend application will launch and open automatically at <http://localhost:3000/>.

D.4 System Operation & User Guide:

➤ Authentication & User Registration

- **Sign Up / Login:** Navigate to <http://localhost:3000/login>. New users can register by selecting their role (Student or Instructor) and providing credentials.
- **JWT Authentication:** Upon successful login, the system issues a Secure JSON Web Token (JWT) to manage user sessions across API requests.

➤ Navigating the Student Dashboard

- **Project Selection:** Once logged in as a student, the dashboard displays available project-based learning paths. Click on any active project to open its workspace.
- **Integrated IDE Workspace:** The workspace is split into three core panels:
 1. *Instructions Panel:* Displays step-by-step task requirements.
 2. *Code Editor:* An in-browser IDE where students write and edit their source code.
 3. *Virtual Branch Control:* Allows users to commit code states or test alternatives safely.

➤ Utilizing the AI Guidance Engine

- While working on a coding task, if a student encounters an error or requires assistance, they can click the "**Get AI Hint**" button.
- The system analyzes the current code snippet, communicates with the backend AI module, and returns a progressive, contextual hint without revealing the direct solution.

References

- ❖ Django Software Foundation (n.d.). *Django Documentation*. <https://docs.djangoproject.com/> (Accessed: July 25, 2026).
- ❖ React Team (n.d.). *React Documentation*. <https://react.dev/> (Accessed: July 25, 2026).
- ❖ Ollama (n.d.). *Ollama Documentation*. <https://ollama.com/> (Accessed: July 25, 2026).
- ❖ GitHub (n.d.). *GitHub Docs*. <https://docs.github.com/> (Accessed: July 25, 2026).
- ❖ SQLite (n.d.). *SQLite Documentation*. <https://www.sqlite.org/docs.html> (Accessed: July 25, 2026).
- ❖ Mozilla Developer Network (n.d.). *MDN Web Docs*.